

AI 제조 기술 관련 교육

『4과목』 리팩토링 + 코드 리뷰 + 배포

2025.12.08-2025.12.19(10일 70시간)

Prepared by Daekyeong K

Ph.D.

Copyright 2022. Daekyeong all rights reserved

SAMSUNG DISPLAY



KORATECH
한국기술교육대학교

『4과목』

리팩토링 + 코드 리뷰 + 배포

- Part 1. 오리엔테이션 및 목표
- Part 2. 리팩토링 이론
- Part 3. 코드 리뷰 + 배포
- Part 3. 종합 실습 & 마무리



학습목표

- 이 워크샵에서는 리팩토링 이론과 기법을 학습하고, AI를 활용해 코드 품질 개선 및 자동 배포 파이프라인을 구축 하는 것에 대해 알 수 있습니다.
- 리팩토링의 정의와 필요성 이해
- 코드 스멜 식별 및 개선 방법 학습
- SOLID 원칙을 실전에 적용
- AI를 활용한 코드 리뷰 자동화
- CI/CD 파이프라인 구축
- Render 플랫폼에 FastAPI 배포

| 기대 효과

- 기능은 동일하지만 더 나은 코드를 작성할 수 있는 능력 습득

절차

● 리팩토링 + 코드 리뷰 + 배포 자동화

순서	주제	학습 핵심 목표
1	리팩토링 이론	웹 동작 원리 이해 및 서비스 요구사항 문서화
2	실습 - 코드 개선	코드 품질 원칙 학습 및 PRD → 코드/테스트 구현
3	배포 자동화	구현된 코드에서 ERD·API·아키텍처 자동 생성

『4과목』

리팩토링 + 코드 리뷰 + 배포

- Part 1. 오리엔테이션 및 목표
- Part 2. 리팩토링 이론
- Part 3. 코드 리뷰 + 배포
- Part 3. 종합 실습 & 마무리



1. 리팩토링이란?

정의

- "외부 동작은 그대로 유지하면서 내부 구조를 개선하는 작업"

Martin Fowler, "Refactoring"

<https://www.youtube.com/watch?v=mNPpfB8JSIU>



Martin Fowler
ThoughtWorks

"Workflows of Refactoring"



1. 리팩토링이란?

| 핵심 개념

- 기능 변경 ❌ → 구조 개선 ✅

정의 :

리팩토링은 외부 동작을 바꾸지 않으면서 내부 구조를 개선하는 방법으로, 소프트웨어 시스템을 변경하는 프로세스이다. 이것은 버그가 끼어 들 가능성을 최소화하면서 코드를 정리하는 정형화된 방법이다

- William Opdyke. Refactoring Object-Oriented Frameworks. University of Illinois at Urbana-Champaign, 1992

1. 리팩토링이란?

리팩토링을 꺼리는 이유

기술적 관점 (Technical Perspective)

원인	설명
테스트 부재 (Lack of Tests)	기존 코드에 단위 테스트나 자동화 테스트가 없으면, 리팩토링 시 동작 보장이 어렵다. “깨질지도 모른다”는 불안감이 생김.
복잡한 의존성 구조 (Complex Dependencies)	모듈 간 의존이 얽혀 있어 코드 변경 시 연쇄 오류가 발생할 위험이 큼.
레거시 기술 (Legacy Stack)	오래된 언어나 프레임워크는 문서나 지원이 부족해 리팩토링이 오히려 더 위험하게 느껴짐.
자동화 도구 부족	정적 분석, 코드 커버리지, 린트 도구 등 품질 보조 툴이 부재.
성능 우려 (Performance Regression)	코드 정리 과정에서 최적화된 로직이 비효율적으로 바뀔까 두려움.

1. 리팩토링이란?

리팩토링을 꺼리는 이유

개발자 심리적 관점 (Psychological Perspective)

원인	설명
기존 코드에 대한 두려움 (Fear of Breaking Things)	"잘 돌아가고 있는데 건드리면 망가질까 봐"라는 심리적 저항.
자기 코드 애착 (Code Ownership Bias)	자신이 쓴 코드에 대한 애착으로, 외부 피드백이나 변경을 방어적으로 받아들임.
보상 구조의 부재	리팩토링은 눈에 보이는 '새 기능'이 아니기 때문에 성과로 인정받기 어려움.
시간 압박 (Time Pressure)	리팩토링보다 기능 출시 속도가 우선시됨.
안정성에 대한 심리적 착각 (Illusion of Stability)	"지금 돌아가니까 굳이 바꿀 필요 없어."라는 현상 유지 심리.

1. 리팩토링이란?

리팩토링을 꺼리는 이유

조직적 / 문화적 관점 (Organizational Perspective)

원인	설명
경영진의 ROI 인식 부족	리팩토링은 즉각적인 수익이 없으므로, 경영진이 시간 낭비로 인식.
프로젝트 일정 압박	일정 지연을 피하기 위해 “기능 우선, 품질 후순위” 문화가 형성.
책임 회피 문화 (Blame Culture)	코드 변경 후 문제 발생 시 개인에게 책임이 돌아오기 때문에 “안 건드리는 게 상책”이라는 분위기.
코드 리뷰 부재	코드 품질에 대한 팀 차원의 관심이 낮아, 리팩토링이 필요하다는 인식 자체가 약함.
기술 부채 축적을 ‘당연시’하는 문화	“나중에 정리하자”가 반복되어 리팩토링이 항상 미뤄짐.



1. 리팩토링이란?

리팩토링을 꺼리는 이유

비즈니스 / 경영적 관점 (Business Perspective)

원인	설명
단기적 수익 중심 사고	리팩토링은 장기적 품질 개선이지만, 경영진은 단기 매출 KPI를 우선시함.
기능 추가 우선 문화 (Feature-Driven)	“새로운 기능이 곧 가치”라는 인식 때문에 유지보수 작업은 뒷전.
리스크 회피 경영 (Risk Aversion)	“새 코드 = 새로운 위험”으로 간주되어 변경 자체를 꺼림.
리팩토링 ROI 측정의 어려움	코드 품질 개선이 수치화되기 어렵고, 투자 대비 효과를 설명하기 힘들.



1. 리팩토링이란?

리팩토링 주저의 결과 결과

결과	설명
기술 부채(Technical Debt) 누적	리팩토링을 미루면 코드 복잡도와 유지보수 비용이 기하급수적으로 증가.
버그 수정 비용 상승	변경이 어려워지면서 사소한 수정에도 시스템 전체가 흔들림.
신규 기능 개발 속도 저하	복잡한 레거시 때문에 새 기능을 붙이기가 어려워짐.
인력 교체 시 리스크 급증	신규 개발자가 코드 구조를 이해하는 데 시간이 오래 걸림.



1. 리팩토링이란?

| 예시

Before: 복잡한 로직

```
def process_order(data):  
    if data['status'] == 'pending' and data['amount'] > 0:  
        # 50줄의 복잡한 로직...  
    return result
```

After: 명확한 구조

```
def process_order(data):  
    validate_order(data)  
    calculate_total(data)  
    update_inventory(data)  
    return create_response(data)
```



1. 리팩토링이란?

리팩토링 vs 재작성

주의사항

"리팩토링은 기능 추가가 아닙니다"

새 기능 추가 → 별도 브랜치에서 작업
리팩토링 → 기존 기능 유지하며 개선

구분	리팩토링	재작성
목적	구조 개선	완전히 새로 만들기
기능	변경 없음	변경 가능
위험도	낮음 (테스트로 검증)	높음
시간	점진적 (일/주 단위)	장기간 (월 단위)
테스트	기존 테스트 활용	새로 작성 필요



2. 왜 리팩토링이 필요한가?

코드 부채(Technical Debt) 감소

빠른 개발 → 복잡한 코드 → 유지보수 어려움 → 생산성 저하

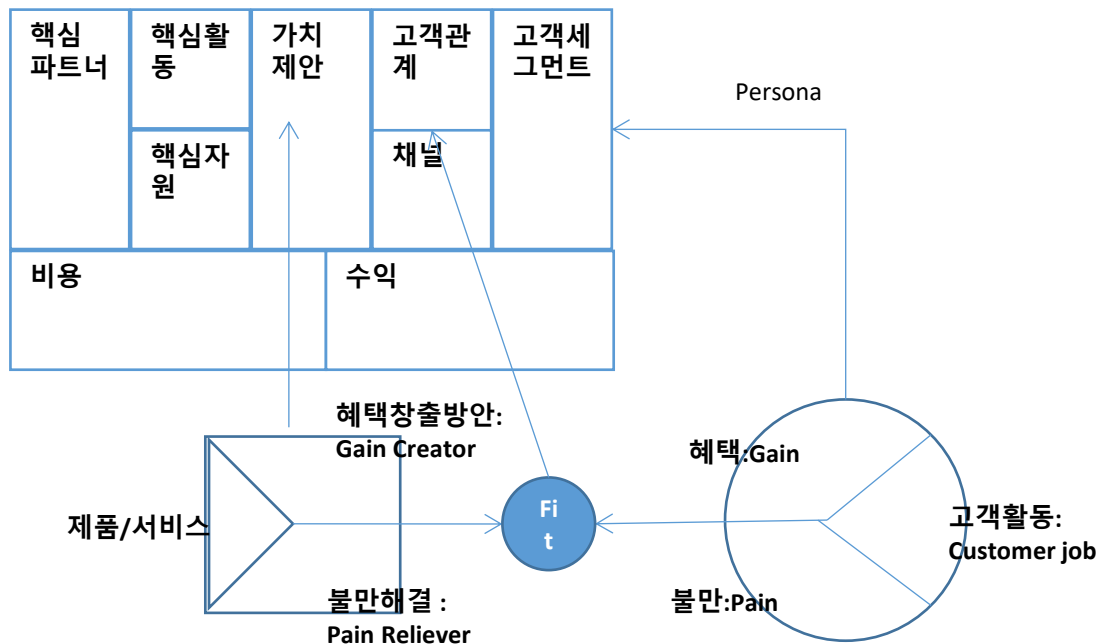
[켄트 벡] "두 개의 모자 비유(The Two Hats Metaphor)"

기능을 추가하는 것
리팩토링을 하는 것

구분하자

빠른 가치 제공

코드의 기술적 부채(Technical Debt) 해결





2. 왜 리팩토링이 필요한가?

| 가독성 향상

이해하기 어려운 코드

```
x = [i for i in d if i['s'] == 'a' and i['p'] > 100]
```

명확한 코드

```
active_premium_orders = [  
    order for order in orders  
    if order['status'] == 'active' and order['price'] > 100  
]
```



2. 왜 리팩토링이 필요한가?



| 확장성 확보

새 기능 추가가 쉬워짐
버그 수정 시간 단축
팀 협업 효율성 증가



2. 왜 리팩토링이 필요한가?

Refactoring 중요성

바르게 가는 것이 가장 빨리 가는 방법이다

- 클릭 아키텍처 책(로버트 C. 마틴(Robert C. Martin), 흔히 "Unde Bob") 중 -

- 소프트웨어 개발 과정에서 "올바른 설계 원칙과 방법론을 따르는 것이 결국 가장 빠르고 효율적인 길"
- 이미 잘 동작하는 코드를 반복해서 고치는 것이 비효율적인 일이라고 생각할 수도 있다.
- 그러나 제품이 일정규모 이상으로 커지게 되면 클린하지 않은 코드들로 새로운 기능 개발이 어렵고, 더 오래 걸리게 된다.
- 그러나 정리된 코드 기반에서 새로운 기능을 개발하게 되면 훨씬 빠르게 개발할 수 있다.
- 결국, 리팩토링을 하는 이유는 그것이 적은 비용으로 훨씬 빠르게 개발할 수 있도록 하기 때문이다.



3. 리팩토링의 전제 조건

지침

"Vision without execution is just a dream. Execution without vision is a nightmare."(비전 없는 실행은 악몽이고, 실행 없는 비전은 꿈일 뿐이다.)

- 넓게 바라보는 비전과 실행 가능한 준비가 동시에 필요하다는 것을 강조하는 경구입니다.

"Inspect and Adapt" (점검하고 적응하라)

- 넓게 관찰하고 현재 상황을 평가한 후, 실행 가능한 방법을 조정하여 개선해야 함을 강조합니다.
- Agile 및 DevOps의 핵심 철학입니다.



3. 리팩토링의 전제 조건

테스트 커버리지

리팩토링 전 필수 확인

```
pytest --cov=app tests/
```

Coverage should be > 80%



3. 리팩토링의 전제 조건

버전 관리

```
git checkout -b refactor/improve-order-service
```

```
# 작은 단위로 커밋
```

```
git commit -m "Extract payment validation logic"
```



3. 리팩토링의 전제 조건

안전한 환경

개발 환경에서 먼저 테스트

코드 리뷰 진행

단계별 배포 (Staging → Production)

경고

"테스트 없이 리팩토링하지 마세요!"

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

돈 로버츠(Don Roberts)의 "3의 법칙(The Rule of Three)"

단계	설명	Python 예시
처음엔 단순하게	중복이 없으면 그냥 구현	<code>get_user_full_name()</code>
두 번째 중복은 무시	아직 리팩토링하지 않음	<code>get_admin_full_name()</code> 추가
세 번째부터 리팩토링	공통화 / 추상화	<code>_format_full_name()</code> 또는 <code>NameFormatter</code> 로 분리

3. 리팩토링의 전제 조건

| Refactoring은 언제 할까?

돈 로버츠(Don Roberts)의 "3의 법칙(The Rule of Three)"

1단계: 처음에는 그냥 한다 (단순 구현)

user_service.py

```
class UserService:
```

```
    def get_user_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name
```

설명:

- 처음에는 단 하나의 기능만 있으므로 단순하게 구현합니다.
- “사용자 이름 + 성”을 합쳐서 문자열로 반환합니다.
- 이 시점에서는 **중복 코드가 없기 때문에 리팩토링 필요 없음.**

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

돈 로버츠(Don Roberts)의 "3의 법칙(The Rule of Three)"

2단계: 비슷한 일을 두 번째로 하게 되면 일단 진행한다 (중복 발생)

user_service.py

```
class UserService:
```

```
    def get_user_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name
```

```
    def get_admin_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name
```

설명:

- 이제 관리자(Admin) 이름을 위한 메서드도 필요해서 비슷한 기능을 복사했습니다.
- 중복이 생겼지만, 이 시점에서는 기능 구현을 우선하기 때문에 리팩토링을 미룹니다.
- 이런 단계는 “빠른 피드백 확보” 또는 “**실험적 구현 단계**”에서 흔히 일어납니다.



3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

돈 로버츠(Don Roberts)의 "3의 법칙(The Rule of Three)"

3단계: 세 번째부터는 리팩토링 (공통화)

user_service.py

```
class UserService:
```

```
    def _format_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name
```

```
    def get_user_full_name(self, first_name: str, last_name: str) -> str:
        return self._format_full_name(first_name, last_name)
```

```
    def get_admin_full_name(self, first_name: str, last_name: str) -> str:
        return self._format_full_name(first_name, last_name)
```

설명:

- 중복된 로직(first_name + " " + last_name)을 하나의 내부 메서드로 추출.
- _format_full_name()은 재사용 가능한 "이름 포매팅 로직"이 됩니다.
- 이제 유지보수가 쉬워졌고, 형식이 바뀌더라도 한 곳만 수정하면 됩니다.
- 즉, 중복 제거(DRY: Don't Repeat Yourself) 원칙을 적용한 상태입니다.

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

돈 로버츠(Don Roberts)의 "3의 법칙(The Rule of Three)"

4단계: 공통 로직을 별도 유틸리티 클래스로 이동

name_formatter.py

```
class NameFormatter:
    @staticmethod
    def format_full_name(first_name: str, last_name: str) -> str:
        return first_name + " " + last_name
```

user_service.py

```
from name_formatter import NameFormatter
```

```
class UserService:
    def get_user_full_name(self, first_name: str, last_name: str) -> str:
        return NameFormatter.format_full_name(first_name, last_name)

    def get_admin_full_name(self, first_name: str, last_name: str) -> str:
        return NameFormatter.format_full_name(first_name, last_name)
```

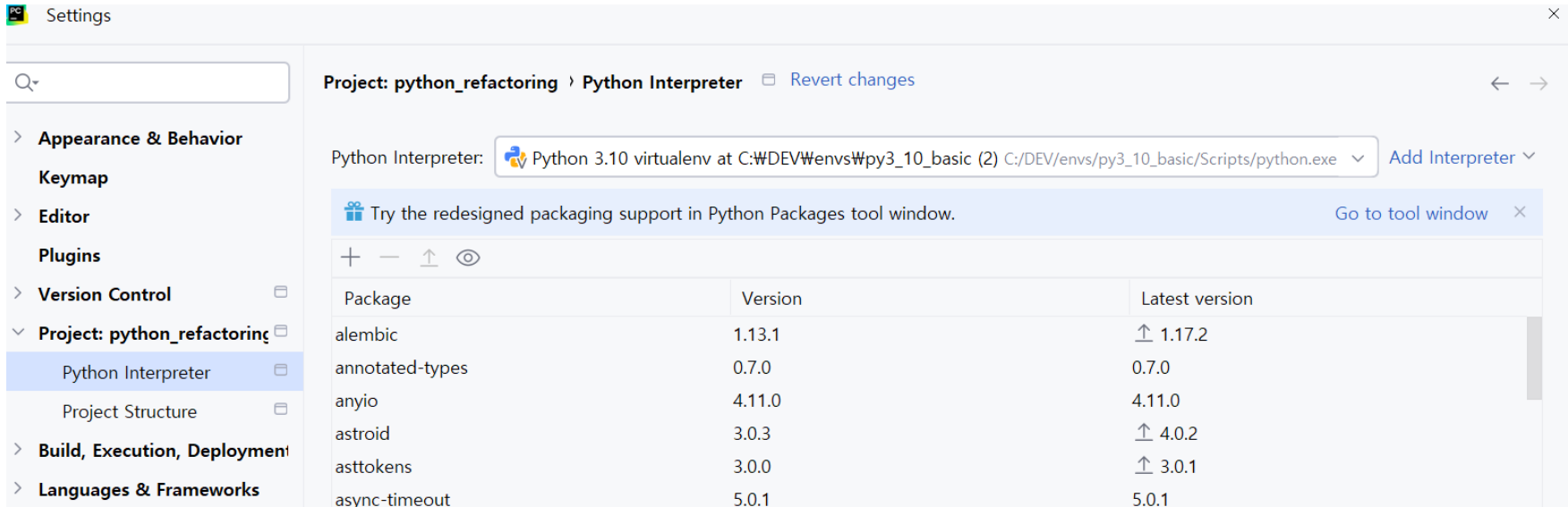
설명:

- 여러 클래스(User, Admin 등)에서 동일한 이름 포매팅 규칙을 사용할 수 있도록 별도의 유틸리티 클래스 **NameFormatter**로 분리했습니다.
- 이렇게 하면 "이름 포맷 변경" 같은 공통 규칙을 한 곳에서 통제할 수 있습니다.
- Java의 static 메서드처럼, Python에서는 @staticmethod를 사용해 객체 생성 없이 직접 호출 가능합니다.

3. 리팩토링의 전제 조건

3의 법칙(The Rule of Three) 실습

C:\DEV\SamsungDisplay\Week_2\Code_pro\python_refactoring



The screenshot shows the PyCharm Settings window for the project 'python_refactoring'. The 'Python Interpreter' section is selected, showing the current interpreter as 'Python 3.10 virtualenv at C:\DEV\envs\py3_10_basic (2) C:\DEV\envs\py3_10_basic\Scripts\python.exe'. Below this, a table lists installed packages and their latest versions.

Package	Version	Latest version
alembic	1.13.1	1.17.2
annotated-types	0.7.0	0.7.0
anyio	4.11.0	4.11.0
astroid	3.0.3	4.0.2
asttokens	3.0.0	3.0.1
async-timeout	5.0.1	5.0.1



3. 리팩토링의 전제 조건

user_service-1.py

```
class UserService:
    def get_user_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name

# 동작 테스트용 코드
if __name__ == "__main__":
    service = UserService()

    # 입력값
    first = "Gil-Dong"
    last = "Hong"

    # 메서드 호출
    full_name = service.get_user_full_name(first, last)

    print("Full Name:", full_name)
```



3. 리팩토링의 전제 조건

user_service-2.py

```
class UserService:
    def get_user_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name

    def get_admin_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name

# 실행 테스트
if __name__ == "__main__":
    service = UserService()

    # 사용자 이름 테스트
    user_full_name = service.get_user_full_name("Dae-Kyung", "Kim")
    print("User Full Name:", user_full_name)

    # 관리자 이름 테스트
    admin_full_name = service.get_admin_full_name("Admin", "Kim")
    print("Admin Full Name:", admin_full_name)
```



3. 리팩토링의 전제 조건

user_service-3.py

```
class UserService:
    def _format_full_name(self, first_name: str, last_name: str) -> str:
        return first_name + " " + last_name

    def get_user_full_name(self, first_name: str, last_name: str) -> str:
        return self._format_full_name(first_name, last_name)

    def get_admin_full_name(self, first_name: str, last_name: str) -> str:
        return self._format_full_name(first_name, last_name)

# 실행 테스트
if __name__ == "__main__":
    service = UserService()

    # 사용자 이름 테스트
    print(service.get_user_full_name("Dae-Kyung", "Kim"))

    # 관리자 이름 테스트
    print(service.get_admin_full_name("Admin", "Lee"))
```




3. 리팩토링의 전제 조건



1. name_formatter.py (name_formatter_4.py)

```
# name_formatter.py
```

```
class NameFormatter:
```

```
    @staticmethod
```

```
    def format_full_name(first_name: str, last_name: str) -> str:
```

```
        return first_name + " " + last_name
```



3. 리팩토링의 전제 조건



2. user_service.py (user_service_4.py)

```
# user_service.py
```

```
from name_formatter_4 import NameFormatter
```

```
class UserService:
```

```
    def get_user_full_name(self, first_name: str, last_name: str) -> str:  
        return NameFormatter.format_full_name(first_name, last_name)
```

```
    def get_admin_full_name(self, first_name: str, last_name: str) -> str:  
        return NameFormatter.format_full_name(first_name, last_name)
```



3. 리팩토링의 전제 조건



3. main.py (실행용 예제)

```
# main.py
```

```
from user_service_4 import UserService
```

```
if __name__ == "__main__":  
    service = UserService()
```

```
    print(service.get_user_full_name("Dae-Kyung", "Kim"))  
    print(service.get_admin_full_name("Admin", "Lee"))
```

```
C:\DEV\envs\py3_10_basic\Scripts\python.exe C:\DEV\SamsungDisplay\Week_2\Code_pro\python_refactoring\main.py
```

```
Dae-Kyung Kim
```

```
Admin Lee
```

```
Process finished with exit code 0
```



3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

- 코드베이스에 산재한 작고 명확한 문제들을 점진적으로 해결하여 코드의 품질을 서서히 향상시키는 것

예제 1: 불필요한 코드 제거-order_processor.py

Before:

```
class OrderProcessor:
    def calculate_total(self, price: float, quantity: int, discount: float) -> float:
        total = 0

        if discount > 0:
            total = (price * quantity) - discount
        else:
            total = price * quantity

        # 불필요한 디버깅용 출력
        print(f"Total after discount: {total}")

        return total
```



이 코드의 문제점

print() 문은 디버깅용으로만 필요하고,
실제 서비스 코드에서는 로그 시스템으로 대체되
거나 제거되어야 함.

if/else 문이 중복된 계산 로직(price * quantity)을
포함 → 가독성 저하.

“할인 적용 여부” 로직을 더 단순하게 표현할 수 있
음.

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

- 코드베이스에 산재한 작고 명확한 문제들을 점진적으로 해결하여 코드의 품질을 서서히 향상시키는 것

예제 1: 불필요한 코드 제거-order_processor.py

After:

```
class OrderProcessor:
    def calculate_total(self, price: float, quantity: int, discount: float) -> float:
        total = price * quantity

        if discount > 0:
            total -= discount

    return total
```

변경 항목	설명
디버깅 코드 제거	print() 문 제거 → 실제 서비스 코드에서는 불필요함
조건문 단순화	if discount > 0 블록만 남기고, 중복된 계산 제거
가독성 향상	한눈에 "가격 × 수량 → 할인 차감 → 반환" 흐름이 보임
유지보수 용이성 향상	나중에 로깅 시스템(logging)이나 추가 정책을 쉽게 붙일 수 있음

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 2: 중복 코드 정리

Before:

```
def send_email_to_user(user_email):  
    print(f"Sending email to user: {user_email}")  
  
def send_email_to_admin(admin_email):  
    print(f"Sending email to admin: {admin_email}")
```

문제점:

두 함수 모두 이메일을 보내는 기능이 동일한데, 수신자 종류(user, admin)만 다름 → 중복 코드 발생





3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 2: 중복 코드 정리

After:

```
class EmailSender:
    def send_email(self, email, recipient_type):
        print(f'Sending email to {recipient_type}: {email}')
```

개선점:

- send_email_to_user()와 send_email_to_admin()을 하나로 통합
- 수신자 구분은 recipient_type 인자("user", "admin")로 처리
- 코드 중복 제거, 유지보수 용이

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 3: 의미 없는 변수 및 로직 제거

Before:

```
class AverageCalculator:
    def calculate_average(self, numbers):
        total_sum = 0
        for number in numbers:
            total_sum += number

        total_count = len(numbers)
        average_value = float(total_sum) / total_count # 불필요한 중간 변수
        return average_value
```

문제점:

average_value 변수는 한 번만 사용되고, 의미가 중복됨

단순 계산 결과를 저장하기 때문에 불필요한 변수 선언



3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 3: 의미 없는 변수 및 로직 제거

After:

```
class AverageCalculator:
    def calculate_average(self, numbers):
        total_sum = 0
        for number in numbers:
            total_sum += number
        return total_sum / len(numbers)
```

개선점:

- 불필요한 변수 `average_value` 제거
- 계산식을 `return` 문에서 바로 표현하여 더 간결하고 직관적
- 코드의 가독성, 유지보수성 향상



3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 4. 사용되지 않는 코드 제거

Before:

```
class PriceCalculator:
    def calculate_total_price(self, price, quantity):
        return price * quantity

    def unused_function(self):
        print("This function is not used.")
```



문제점:

unused_function()은 프로그램 어디에서도 호출되지 않음

의미 없는 코드가 유지보수성과 가독성을 떨어뜨림

이런 함수는 **“죽은 코드(dead code)”**로 간주됨



3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"쓰레기 줍기 리팩토링(Garbage Collecting Refactoring)"

예제 4. 사용되지 않는 코드 제거

After:

```
class PriceCalculator:
    def calculate_total_price(self, price, quantity):
        return price * quantity
```

개선점:

사용되지 않는 `unused_function()` 메서드를 삭제

코드가 더 간결하고 깨끗하게(clean code) 됨

클래스의 책임이 명확해지고, 유지보수성이 향상됨

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"이해하기 위한 리팩토링 (Refactoring to Understand)"

Before:

```
class CommissionCalculator:  
    def a(self, x, y):  
        return x * 0.05 * y / (x + y)
```

문제점:

함수명 a, 변수명 x, y는 의미가 불분명합니다.

코드의 의도("판매 수수료율 계산")를 한눈에 파악하기 어렵습니다.

유지보수 시 다른 개발자가 이해하기 힘듭니다.



3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"이해하기 위한 리팩토링 (Refactoring to Understand)"

After:

```
class CommissionCalculator:
    def calculate_commission_rate(self, sales, expenses):
        return sales * 0.05 * expenses / (sales + expenses)
```

개선점:

a() → calculate_commission_rate()로 변경 → 기능
목적이 명확

변수 x, y → sales, expenses로 의미 부여

코드의 계산 구조를 그대로 유지하면서 이해하기 쉬운
형태로 개선

3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"변화를 위한 리팩토링 (Refactoring to Change)"

Before:

```
class OrderProcessor:
    def process_order(self, order_type):
        if order_type == "online":
            print("Processing online order")
        elif order_type == "in_store":
            print("Processing in-store order")
        else:
            print("Unknown order type")
```

문제점:

새로운 주문 유형("subscription" 등)이 추가될 때
마다

if-elif 조건문을 계속 추가해야 함 → OCP(Open-Closed Principle) 위반

코드 수정 범위가 넓고, 유지보수성이 낮음



3. 리팩토링의 전제 조건

Refactoring은 언제 할까?

"변화를 위한 리팩토링 (Refactoring to Change)"

After:

```
class OrderProcessor:
    def __init__(self):
        self.order_processors = {
            "online": self._process_online_order,
            "in_store": self._process_in_store_order
        }

    def process_order(self, order_type):
        processor = self.order_processors.get(order_type)
        if processor:
            processor()
        else:
            print("Unknown order type")

    def _process_online_order(self):
        print("Processing online order")

    def _process_in_store_order(self):
        print("Processing in-store order")
```

개선점:

주문 유형별 처리 로직을 딕셔너리(Map)로 등록

새로운 주문 유형이 생기면 단순히
self.order_processors에 추가만 하면 됨

조건문 제거, 확장에 유연한 구조

코드가 **OCP(Open-Closed Principle)**에 부합

3. 리팩토링의 전제 조건

어떻게 리팩토링 하는가?

Find some code that **"smells"**



Determine how to **simplify** this code



Make the **simplifications**



Run tests to ensure things still work correctly



Repeat the simplify/test cycle until the smell is gone





3. 리팩토링의 전제 조건

어떻게 리팩토링 하는가?

앞 서 그림에서 제공한 리팩토링 절차의 문제점

- 테스트 실행이 리팩토링 후에 진행됨
- 그림에서는 코드를 단순화한 후(Make the simplifications) 테스트를 실행(Run tests)하도록 되어 있음.
- 그러나, 리팩토링을 시작하기 전에 먼저 테스트를 실행해야 안전한 리팩토링이 가능.
- 리팩토링 전, 기존 코드의 정상 동작 여부를 보장할 수 없음
- 리팩토링 전에 코드가 올바르게 동작하는지 확인하지 않으면, 리팩토링 과정에서 기능이 깨져도 이를 알 수 없음.
- 따라서, 리팩토링 전 반드시 기존 코드가 테스트를 통과하는지 확인하는 과정이 필요.

3. 리팩토링의 전제 조건

어떻게 리팩토링 하는가?

앞 선 그림에서 제시된 절차를 테스트 우선(TDD) 원칙을 반영하여 보완하면 아래와 같이 개선할 수 있습니다.

1. 테스트가 존재하는지 확인 (Test First)
 - 기존 코드가 정상 동작하는지 테스트를 먼저 실행하여 확인
 - 테스트가 없다면, 기본적인 단위 테스트를 먼저 작성
2. 코드 스멜(Code Smell) 감지
 - 유지보수성이 떨어지는 코드(중복 코드, 긴 함수, 불필요한 매개변수 등)를 찾아 개선할 부분을 결정
3. 리팩토링 수행 (Refactor)
 - 코드 구조를 단순화하고, 중복을 제거하며 가독성을 향상
4. 테스트 실행 (Test Again!)
 - 리팩토링 후에도 기능이 정상적으로 동작하는지 테스트 실행
 - 만약 테스트가 실패하면, 리팩토링 과정에서 실수가 있었음을 의미
 - 필요하면 롤백하거나 다시 수정
5. 리팩토링 반복
 - 코드 스멜이 완전히 사라질 때까지 "리팩토링 → 테스트 실행" 반복

3. 리팩토링의 전제 조건

어떻게 리팩토링 하는가?

앞 선 그림과 보완된 절차 비교

기존 그림의 절차	보완된 안전한 리팩토링 절차
1. 코드 스멜 찾기	1. 테스트가 있는지 확인 (Test First)
2. 코드 단순화 결정	2. 코드 스멜 감지
3. 코드 단순화 수행	3. 리팩토링 수행
4. 테스트 실행	4. 테스트 실행 (Test Again!)
5. 반복 수행	5. 리팩토링 반복

1. 질문(Ask):테스트 작성을 통해 시스템에 질문한다. (테스트 수행 결과는 실패)
2. 응답(Respond):테스트를 통과하는 코드를 작성해서 질문에 대답한다. (테스트 성공)
3. 정제(Refine,Refactoring): 아이디어를 통합하고, 불필요한 것은 제거하고, 모호한 것은 명확히 해서 대답을 정제한다. (리팩토링)
4. 반복(Repeat): 다음 질문을 통해 대화를 계속 진행한다.



3. 리팩토링의 전제 조건

코드의 어느 부분을 리팩토링 해야 할까?

1. 기이한 이름 (Mysterious Name)
2. 중복 코드 (Duplicated Code)
3. 긴 함수 (Long Function)
4. 긴 매개변수 목록 (Long Parameter List)
5. 전역 데이터 (Global Data)
6. 가변 데이터 (Mutable Data)
7. 뒤텀킨 변경 (Divergent Change)
8. 산탄총 수술 (Shotgun Surgery)
9. 기능 편애 (Feature Envy)
10. 데이터 뭉치 (Data Clumps)
11. 기본형 집착 (Primitive Obsession)
12. 반복되는 스위치문 (Repeated Switches)
13. 반복문 (Loops)
14. 성의 없는 요소 (Lazy Element)
15. 추측성 일반화 (Speculative Generality)
16. 임시 필드 (Temporary Field)
17. 메시지 체인 (Message Chains)
18. 중재자 (Middle Man)
19. 내부자 거래 (Insider Trading)
20. 거대한 클래스 (Large Class)
21. 서로 다른 인터페이스의 대안 클래스들
(Alternative Classes with Different Interfaces)
22. 데이터 클래스 (Data Class)
23. 상속 포기 (Refused Bequest)
24. 주석 (Comments)



3. 리팩토링의 전제 조건

리팩토링과 리스트럭처링

- Refactoring (리팩토링) : 코드의 내부 구조를 개선하지만, 외부 동작(기능)은 그대로 유지
- Restructuring (리스트럭처링) : 시스템 또는 모듈의 전체 구조/설계를 재편하는 더 큰 범위의 변경



3. 리팩토링의 전제 조건

리팩토링과 리스트럭처링

리팩토링 예제 (작은 수준의 개선)

전: 중복된 로직

```
def calculate_discount(price, is_vip):  
    if is_vip:  
        return price - (price * 10 / 100)  
    else:  
        return price - (price * 5 / 100)
```

- if문 안팎의 계산식이 거의 동일 ($\text{price} - (\text{price} * X / 100)$)
- 할인율(10 또는 5)만 다름
- 불필요한 중복이 발생해 코드가 장황함

후: 중복 제거 리팩토링

```
def calculate_discount(price, is_vip):  
    discount_rate = 10 if is_vip else 5  
    return price - (price * discount_rate / 100)
```

기능은 동일하지만 코드가 간결하고 유지보수 쉬움

- 할인율만 변수(discount_rate)로 분리
- 조건문 내부 중복 제거, 코드 한 줄로 단순화
- 기능은 동일하지만 가독성 향상, 유지보수 용이



3. 리팩토링의 전제 조건

리팩토링과 리스트럭처링

리스트럭처링 예제 (구조적 재조정)

- 전: 할인 관련 로직이 Order 클래스에 섞여 있음

```
def calculate_discount(price, is_vip):  
    if is_vip:  
        return price - (price * 10 / 100)  
    else:  
        return price - (price * 5 / 100)
```

두 조건문 모두 $\text{price} - (\text{price} * \dots / 100)$ 구조가 완전히 동일함

단지 할인율만 다르므로 중복이 불필요

새로운 할인 조건이 추가되면 if-elif 문이 계속 늘어남 → 유지보수 어려움

- 후: 구조 재조정 (리스트럭처링)

```
def calculate_discount(price, is_vip):  
    discount_rate = 10 if is_vip else 5  
    return price - (price * discount_rate / 100)
```

개선점:

할인율(10% 또는 5%)만 변수로 분리

if문 내부의 중복된 계산식을 제거

코드가 더 짧고 직관적이며, 유지보수가 쉬움

3. 리팩토링의 전제 조건

리팩토링과 리스트럭처링

Refactoring (리팩토링) 에, Restructuring (리스트럭처링) 을 포함합니다.

구분	리팩토링	리스트럭처링
범위	코드 내부 한 함수/클래스	시스템 또는 아키텍처 수준
목적	코드 품질 향상, 가독성, 유지보수성	구조 재편성, 책임 분리, 모듈화
예시	중복 제거, 변수 이름 변경	클래스 분리, 패키지 구조 이동

4. 코드 스멜과 리팩토링

| 코드 스멜(Code Smell)이란?

"버그는 아니지만 개선이 필요한 코드의 징후"
주요 특징

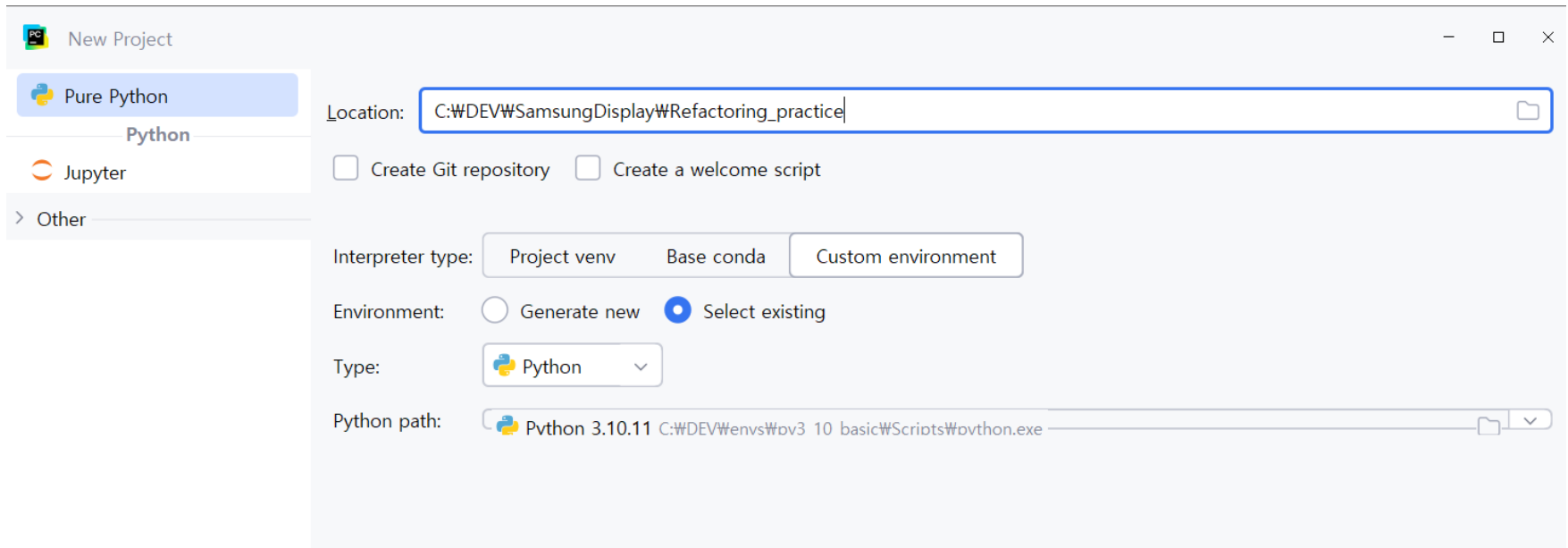
버그는 아니지만 문제의 신호
유지보수성 저하
리팩토링 필요성 암시

```
# Code Smell: Long Method (긴 메서드)
def create_order(self, data):
    # 100줄 이상의 로직
    # 검증, 계산, 저장, 알림 등 모두 포함
    ...
    return result # 라인 120
```

4. 코드 스멜과 리팩토링

코드 스멜(Code Smell)이란?

C:\DEV\Code_pro\Refactoring_practice



The screenshot shows the 'New Project' dialog in Visual Studio Code. The left sidebar lists project types: 'Pure Python' (selected), 'Python' (with a sub-option for 'Jupyter'), and 'Other'. The main area contains the following fields and options:

- Location:** A text box containing the path `C:\DEV\SamsungDisplay\Refactoring_practice`.
- Options:** Two checkboxes, 'Create Git repository' and 'Create a welcome script', both of which are unchecked.
- Interpreter type:** Three buttons: 'Project venv' (selected), 'Base conda', and 'Custom environment'.
- Environment:** Two radio buttons: 'Generate new' (unchecked) and 'Select existing' (checked).
- Type:** A dropdown menu showing 'Python'.
- Python path:** A text box showing the path `C:\DEV\envs\py3_10\basic\Scripts\python.exe` for 'Python 3.10.11'.

코드 스멜 유형 1 - MYSTERIOUS NAME

ChangeFunctionDeclaration.py

< Before >

```
def calculate_sum(a, b):  
    return a + b
```

문제점:

- 함수 이름 calculate_sum은 모호함 — “무엇의 합을 계산하는지” 불분명
- 향후 기능 확장(예: 세 개의 수를 더함)을 고려하지 않음

| 코드 스멜 유형 1 - MYSTERIOUS NAME

ChangeFunctionDeclaration.py

< After >

```
def sum_numbers(a, b, c=0):  
    return a + b + c
```

개선점:

- 함수 이름을 sum_numbers로 변경하여 의도 명확화 (“숫자들의 합 계산”)
- 선택적 매개변수 c 추가로 확장성 확보
- 코드 호출부 수정 최소화 (c가 없으면 기본값 0 사용)

4. 코드 스멜과 리팩토링

코드 스멜 유형 1 - MYSTERIOUS NAME

ChangeFunctionDeclaration.py

PyCharm에서 함수 이름과 매개변수 변경하기

1. 함수 이름 (calculate_sum) 위에 커서를 둡니다.

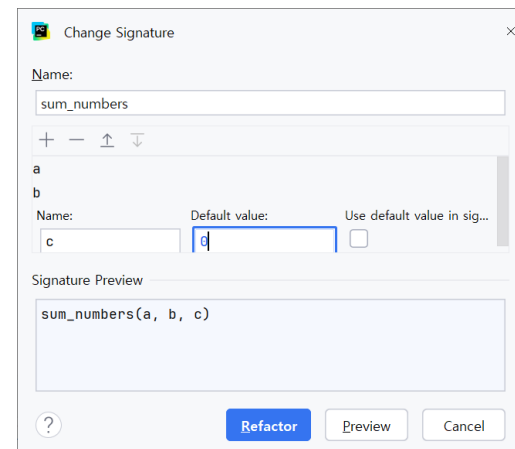
2. Ctrl + F6 (Mac: ⌘ + F6) 을 누릅니다.

3. Change Signature 창이 열리면 →

1. 함수 이름을 sum_numbers로 변경

2. 매개변수 c=0 추가

4. OK 클릭 → 모든 참조가 자동으로 업데이트됩니다





4. 코드 스멜과 리팩토링

코드 스멜 유형 2 - Duplicated Code

- 똑같은 코드 구조가 두 군데 이상 있는 경우
- 한 클래스의 두 메서드 안에 같은 코드가 들어있는 경우
- 한 클래스의 두 하위 클래스에 완벽히 같지는 않지만 유사한 코드를 가지고 있는 경우
- 서로 상관이 없는 두 클래스에 완벽히 같은 코드가 있는 경우
- 사용할 수 있는 리팩토링 기술
 - 동일한 코드를 여러 메소드에서 사용하는 경우, 함수 추출하기 (Extract Function)
 - 코드가 비슷하게 생겼지만 완전히 같지는 않은 경우, 코드 분리하기 (Slide statements)
 - 여러 하위 클래스에 동일한 코드가 있다면, 메소드 올리기 (Pull Up Method)

코드 스멜 유형 2 - Duplicated Code

DuplicatedCode.py

< Before >

```
def print_order_details(order):  
    print("Order Summary:")  
    total = 0  
  
    # 총액 계산 로직 (길고 반복될 가능성 있음)  
    for item in order['items']:  
        total += item['price']  
  
    print("Total Price:", total)
```

문제점:

- 총액 계산 로직이 직접 포함되어 있어 코드가 길어짐
- 동일한 로직을 다른 함수에서도 사용할 가능성 존재 → 중복 발생 위험
- 가독성 저하, 유지보수 어려움

| 코드 스멜 유형 2 - Duplicated Code

DuplicatedCode.py

< After >

```
def calculate_total_price(order):  
    total = 0  
    for item in order['items']:  
        total += item['price']  
    return total
```

```
def print_order_details(order):  
    print("Order Summary:")  
    print("Total Price:", calculate_total_price(order))
```

개선점:

- calculate_total_price()로 총액 계산 부분을 별도 함수로 분리
- 가독성 향상, 중복 제거, 테스트 용이성 증가
- print_order_details()의 역할이 "출력"으로 더 명확해짐

4. 코드 스멜과 리팩토링

코드 스멜 유형 2 - Duplicated Code DuplicatedCode.py

PyCharm에서 실제 리팩토링 순서

1. `print_order_details()` 함수 안의 **총액 계산 부분**을 마우스로 드래그 선택

```
total = 0
for item in order['items']:
    total += item['price']
```

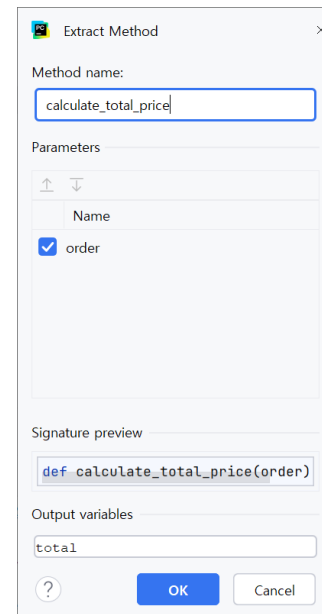
2. 단축키 실행

1. Windows/Linux → Ctrl + Alt + M

2. Mac → ⌘ + ⌥ + M

3. 새 함수 이름 입력 → `calculate_total_price`

4. Refactor 버튼 클릭





4. 코드 스멜과 리팩토링

코드 스멜 유형 3 - Long Method

장황한 메서드; 메서드의 길이가 지나치게 길고 중복된 코드가 많은 코드

- 짧은 함수 vs 긴 함수
- 함수가 길 수록 더 이해하기 어렵다. vs 짧은 함수는 더 많은 문맥 전환을 필요로 한다.
- “과거에는” 작은 함수를 사용하는 경우에 더 많은 서브루틴 호출로 인한 오버헤드가 있었다.
- 작은 함수에 “좋은 이름”을 사용했다면 해당 함수의 코드를 보지 않고도 이해할 수 있다.
- 어떤 코드에 “주석”을 남기고 싶다면, 주석 대신 함수를 만들고 함수의 이름으로 “의도”를 표현해보자.

사용할 수 있는 리팩토링 기술

- 99%는, “함수 추출하기 (Extract Function)”로 해결할 수 있다.
- 함수로 분리하면서 해당 함수로 전달해야 할 매개변수가 많아진다면 다음과 같은 리팩토링을 고려해볼 수 있다.



4. 코드 스멜과 리팩토링

코드 스멜 유형 3 - Long Method

함수로 분리하면서 해당 함수로 전달해야 할 매개변수가 많아진다면 다음과 같은 리팩토링을 고려해볼 수 있다.

- 임시 변수를 질의 함수로 바꾸기 (Replace Temp with Query)
- 매개변수 객체 만들기 (Introduce Parameter Object)
- 객체 통째로 넘기기 (Preserve Whole Object)
- “조건문 분해하기 (Decompose Conditional)”를 사용해 조건문을 분리할 수 있다.

같은 조건으로 여러 개의 Switch 문이 있다면, “조건문을 다형성으로 바꾸기 (Replace Conditional with Polymorphism)”를 사용할 수 있다.

반복문 안에서 여러 작업을 하고 있어서 하나의 메소드로 추출하기 어렵다면, “반복문 쪼개기 (Split Loop)”를 적용할 수 있다.

코드 스멜 유형 3 - Long Method

LongMethod.py

< Before >

class Report:

```
def print_report(self, start_date, end_date, employee_name):  
    print(f"Report for {employee_name}")  
    print(f"From {start_date} to {end_date}")
```

문제점:

- 매개변수가 많아질수록 (start_date, end_date, employee_name, ...) 함수 호출 시 가독성이 떨어짐
- 관련 있는 값들이 따로 전달되어 응집도가 낮음
- 매개변수 추가 시 모든 호출부를 수정해야 함

4. 코드 스멜과 리팩토링

| 코드 스멜 유형 3 - Long Method

LongMethod.py

< After >

```
class ReportPeriod:
    """기간 정보를 묶는 클래스"""
    def __init__(self, start_date, end_date):
        self.start_date = start_date
        self.end_date = end_date

class Report:
    def print_report(self, period: ReportPeriod, employee_name: str):
        print(f"Report for {employee_name}")
        print(f"From {period.start_date} to {period.end_date}")
```

개선점:

- start_date, end_date → ReportPeriod 객체로 묶음
- 매개변수 개수 감소 (3개 → 2개)
- 코드 응집도 향상, 유지보수 용이
- ReportPeriod를 다른 함수에서도 재사용 가능

4. 코드 스멜과 리팩토링

| 코드 스멜 유형 3 - Long Method

LongMethod.py

PyCharm에서 실제 리팩토링 절차

1. print_report()의 매개변수 중 start_date, end_date 두 값을 마우스로 선택
2. 단축키:
 - Windows/Linux → Ctrl + Alt + Shift + T
 - macOS → Ctrl + T
3. 팝업 메뉴에서 “**Extract Parameter Object**” 선택 -> PyCharm Community Edition(무료 버전)에서는 Extract Parameter Object 리팩토링 기능이 기본적으로 제공되지 않습니다. 수동으로 작업.
4. 새 클래스 이름 입력 → ReportPeriod
5. PyCharm이 자동으로 클래스를 생성하고, 함수 매개변수를 해당 객체로 대체합니다

4. 코드 스멜과 리팩토링

| 코드 스멜 유형 4 - Large Class

문제점SRP 위반

하나의 클래스가 너무 많은 책임을 가짐

```
class OrderService:
    """500줄 이상의 거대 클래스"""

    def create_order(self): ...
    def update_order(self): ...
    def cancel_order(self): ...
    def calculate_total(self): ...
    def apply_discount(self): ...
    def process_payment(self): ...
    def validate_payment(self): ...
    def send_email(self): ...
    def send_sms(self): ...
    def create_invoice(self): ...
    def generate_pdf(self): ...
    # ... 20개 이상의 메서드
```



4. 코드 스멜과 리팩토링

코드 스멜 유형 5 - Feature Envy

잘못된 소속; 어떤 메서드가 자신이 속하지 않은 클래스에 더 많이 접근하는 경우

- 부러워하다라는 뜻. 너무 많은 내용을 갖는 것
- 기능 편애 (Feature Envy)는 클래스가 자신이 속한 클래스의 데이터를 사용하는 대신, 다른 클래스의 데이터를 지나치게 자주 참조하거나 사용하는 경우 발생하는 코드 냄새(Code Smell)

문제점

- 응집도 감소
- 결합도 증가.
- 유지보수성 저하

4. 코드 스멜과 리팩토링

| 코드 스멜 유형 5 - Feature Envy

FeatureEnvy.py

< Before >

```
class Address:
```

```
    def __init__(self, street, city, zip_code):
        self.street = street
        self.city = city
        self.zip_code = zip_code
```

```
class Customer:
```

```
    def __init__(self, name, address):
        self.name = name
        self.address = address
```

```
class Invoice:
```

```
    def print_customer_info(self, customer):
        # Address 데이터를 과도하게 직접 사용
        address = customer.address
        print("Customer Address:")
        print(f"{address.street}, {address.city} {address.zip_code}")
```

| 코드 스멜 유형 5 - Feature Envy

FeatureEnvy.py

< Before >

문제점:

- Invoice 클래스가 Address의 내부 필드를 직접 접근하고 있음
- Invoice가 "주소 포매팅 책임"까지 떠맡고 있음
- 이로 인해 "Feature Envy (특정 클래스 데이터에 과도하게 의존)" 발생
- 유지보수 시 결합도가 높아짐 → SRP(단일 책임 원칙) 위반

코드 스멜 유형 5 - Feature Envy

FeatureEnvy.py

< After > (함수 이름 변경 + 매개변수 추가)

```
class Address:
```

```
    def __init__(self, street, city, zip_code):
        self.street = street
        self.city = city
        self.zip_code = zip_code
```

```
    # 주소 포매팅 책임을 Address 내부로 이동
```

```
    def format_address(self):
        return f"{self.street}, {self.city} {self.zip_code}"
```

```
class Customer:
```

```
    def __init__(self, name, address):
        self.name = name
        self.address = address
```

```
class Invoice:
```

```
    def print_customer_info(self, customer):
        print("Customer Address:")
        # Address의 내부 데이터를 직접 접근하지 않고, 메서드 호출만 수행
        print(customer.address.format_address())
```



4. 코드 스멜과 리팩토링



코드 스멜 유형 5 - Feature Envy

FeatureEnvy.py

< After >

개선점:

- 주소 포매팅(format_address)은 Address 클래스의 책임으로 이동
- Invoice는 단지 “메시지를 전달하는 역할”만 수행
- 책임 분리 (SRP) 실현, 결합도 감소, 가독성 향상

| 코드 스멜 유형 5 - Feature Envy

FeatureEnvy.py

PyCharm에서 Move Method 리팩토링 절차

1. 이동할 메서드 (format_address) 또는 Invoice.print_customer_info의 관련 부분 선택
2. 단축키:
 1. Windows/Linux → F6
 2. macOS → F6
3. 대화창에서 이동할 대상 클래스 선택 (예: Address)
4. PyCharm이 자동으로 코드를 옮기고, 호출부를 업데이트해줍니다



4. 코드 스멜과 리팩토링

코드 스멜 유형 6 - DIVERGENT CHANGE

수정의 산발; 한 클래스가 다양한 원인 때문에 다양한 방식으로 자주 수정되는 경우.

어떤 한 모듈이(함수 또는 클래스가) 여러가지 이유로 다양하게 변경되어야 하는 상황

- 예) 새로운 결제 방식을 도입하거나, DB를 변경할 때 동일한 클래스에 여러 메소드를 수정해야 하는 경우

서로 다른 문제는 서로 다른 모듈에서 해결해야 한다.

모듈의 책임이 분리되어 있을수록 해당 문맥을 더 잘 이해할 수 있으며 다른 문제는 신경 쓰지 않아도 된다.

관련 리팩토링 기술

- “단계 쪼개기 (Split Phase)”를 사용해 서로 다른 문맥의 코드를 분리할 수 있다.
- “함수 옮기기 (Move Function)”를 사용해 적절한 모듈로 함수를 옮길 수 있다.
- 여러가지 일이 하나의 함수에 모여 있다면 “함수 추출하기 (Extract Function)”를 사용할 수 있다.
- 모듈이 클래스 단위라면 “클래스 추출하기 (Extract Class)”를 사용해 별도의 클래스로 분리할 수 있다.

코드 스멜 유형 6 - DIVERGENT CHANGE

DIVERGENT_CHANGE.py

< Before >

```
class Employee:
    def __init__(self, name, address, phone_number):
        self.name = name
        self.address = address
        self.phone_number = phone_number

    def get_contact_info(self):
        return f"{self.address}, {self.phone_number}"
```

문제점:

- Employee 클래스가 “직원 정보”와 “연락처 관리” 두 가지 역할을 동시에 수행
- 책임 분리가 안 되어 있음 (SRP 위반)
- 연락처 관련 코드가 다른 클래스에도 필요할 가능성이 있음
- 수정 시 여러 기능에 영향을 줄 수 있는 뒤엉킨 변화(Divergent Change) 발생



4. 코드 스멜과 리팩토링

코드 스멜 유형 6 - DIVERGENT CHANGE

DIVERGENT_CHANGE.py

After (리팩토링 후 - Extract Class)

```
class ContactInfo:
    """연락처 정보만 담당하는 클래스"""
    def __init__(self, address, phone_number):
        self.address = address
        self.phone_number = phone_number

    def get_contact_info(self):
        return f"{self.address}, {self.phone_number}"

class Employee:
    """직원 기본 정보를 담당하는 클래스"""
    def __init__(self, name, contact_info: ContactInfo):
        self.name = name
        self.contact_info = contact_info
```


코드 스멜 유형 6 - DIVERGENT CHANGE

DIVERGENT_CHANGE.py

< After > (함수 이름 변경 + 매개변수 추가)

개선점:

- Employee에서 연락처 관련 책임(contact_info)을 별도 클래스로 추출
- Employee는 “직원 관리”에 집중, ContactInfo는 “연락처 관리”에 집중
- 코드 구조가 단순해지고, 재사용성 및 유지보수성이 향상

코드 스멜 유형 6 - DIVERGENT CHANGE

DIVERGENT_CHANGE.py

PyCharm에서 실제 리팩토링 절차

1. Employee 클래스 안의 address, phone_number, get_contact_info 부분을 마우스로 선택

```
self.address = address
self.phone_number = phone_number

def get_contact_info(self):
    return f"{self.address}, {self.phone_number}"
```

2. 단축키 실행

1. Windows/Linux → Ctrl + Alt + Shift + T
2. Mac → Ctrl + T
3. 팝업에서 **"Extract Class"** 선택
4. 새 클래스 이름 입력 → ContactInfo
5. PyCharm이 자동으로
 1. 새 클래스를 만들고,
 2. 기존 필드/메서드를 이동시키며,
 3. Employee 내 참조를 self.contact_info로 변경해 줍니다

4. 코드 스멜과 리팩토링

코드 스멜 유형 6 - DIVERGENT CHANGE DIVERGENT_CHANGE.py

 Extract Superclass

Extract superclass from:

Superclass name:

Extract superclass with members to directory

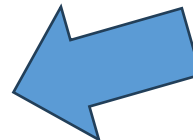
Extract superclass from:

	Member	Make Abstract
☐	  <code>__init__(self, name, address, phone_numbe</code>	
☒	 <code>get_contact_info(self)</code>	☐
☒	 <code>self.address</code>	
☐	 <code>self.name</code>	
☒	 <code>self.phone_number</code>	



```
class Employee:
    def __init__(self, name, address, phone_number):
        self.name = name
        self.address = address
        self.phone_number = phone_number

    def get_contact_info(self):
        return f"{self.address}, {self.phone_number}"
```



Refactor This

- | | | |
|-------------------|------------------------|------------|
| 1 | Change Signature... | Ctrl+F6 |
| Extract/Introduce | | |
| 2 | Introduce Variable... | Ctrl+Alt+V |
| 3 | Introduce Constant... | Ctrl+Alt+C |
| 4 | Introduce Field... | Ctrl+Alt+F |
| 5 | Introduce Parameter... | Ctrl+Alt+P |
| 6 | Extract Method... | Ctrl+Alt+M |
| 7 | Extract Superclass... | |



4. 코드 스멜과 리팩토링

코드 스멜 유형 7 - SHOTGUN SURGERY

기능의 산재; 수정할 때마다 여러 클래스에서 수많은 자잘한 부분을 고쳐야 하는 경우. 산탄총을 쏘는 것

어떤 한 변경 사항이 생겼을 때 여러 모듈을 (여러 함수 또는 여러 클래스를) 수정해야 하는 상황

“뒤엎킨 변경” 냄새와 유사하지만 반대의 상황이다.

- 예) 새로운 결제 방식을 도입하려면 여러 클래스의 코드를 수정해야 한다.

변경 사항이 여러 곳에 흩어진다면 찾아서 고치기도 어렵고 중요한 변경 사항을 놓칠 수 있는 가능성도 생긴다.

관련 리팩토링 기술

- “함수 옮기기 (Move Function)” 또는 “필드 옮기기 (Move Field)”를 사용해서 필요한 변경 내역을 하나의 클래스로 모을 수 있다.
- 비슷한 데이터를 사용하는 여러 함수가 있다면 “여러 함수를 클래스로 묶기 (Combine Functions into Class)”를 사용할 수 있다.
- “단계 쪼개기 (Split Phase)”를 사용해 공통으로 사용되는 함수의 결과물들을 하나로 묶을 수 있다.
- “함수 인라인 (Inline Function)”과 “클래스 인라인 (Inline Class)”로 흩어진 로직을 한 곳으로 모을 수도 있다.

코드 스멜 유형 7 - SHOTGUN SURGERY

SHOTGUN_SURGERY.py

< Before >

```
class Item:
    def __init__(self, price):
        self.price = price

class Order:
    def __init__(self, items):
        self.items = items
        self.base_discount = 0.1 # ✖ Order보다는 Item에 더 관련된 필드

    def calculate_total_price(self):
        total = 0
        for item in self.items:
            total += item.price - (item.price * self.base_discount)
        return total
```

코드 스멜 유형 7 - SHOTGUN SURGERY

SHOTGUN_SURGERY.py

문제점:

- base_discount는 Order가 아닌 Item에 더 관련 있는 속성입니다.
- Order에서 수정 시 다른 여러 메서드가 동시에 변경되어야 하는 산탄총 수술(Shotgun Surgery) 위험이 존재합니다.
- 즉, “할인 관련 로직”이 Item 쪽으로 이동해야 응집도가 높아집니다.

4. 코드 스멜과 리팩토링

코드 스멜 유형 7 - SHOTGUN SURGERY SHOTGUN_SURGERY.py

After (리팩토링 후 - Extract Class)

```
class Item:
    def __init__(self, price, base_discount=0.1):
        self.price = price
        self.base_discount = base_discount # ✅ 필드 이동 완료

    def get_price_with_discount(self):
        return self.price - (self.price * self.base_discount)

class Order:
    def __init__(self, items):
        self.items = items

    def calculate_total_price(self):
        total = 0
        for item in self.items:
            total += item.get_price_with_discount() # ✅ 로직 단순화
        return total
```

4. 코드 스멜과 리팩토링

| 코드 스멜 유형 7 - SHOTGUN SURGERY

SHOTGUN_SURGERY.py

< After > (함수 이름 변경 + 매개변수 추가)

개선점:

- base_discount 필드를 Item 클래스로 이동
- Order는 단순히 Item에게 메시지를 전달하도록 변경
- 응집도↑, 결합도↓, Shotgun Surgery 제거
- 향후 할인 로직 변경 시 Item만 수정하면 됨

코드 스멜 유형 7 - SHOTGUN SURGERY

SHOTGUN_SURGERY.py

PyCharm에서 Move Field 리팩토링 절차

1. Order 클래스 내 base_discount 필드를 커서로 선택
2. 단축키 실행
 1. Windows/Linux → F6
 2. macOS → F6
3. 팝업에서 이동할 대상 클래스 선택 → Item
4. PyCharm이 자동으로
 1. base_discount 필드를 Item으로 이동
 2. 관련 참조를 모두 업데이트합니다



5. 코드 스멜과 리팩토링 기법

대표적인 코드 스멜과 리팩토링 기법

구분	코드 스멜 (Code Smell)	문제점	리팩토링 기법	PyCharm 단축키
①	긴 함수 (Long Function)	한 함수가 너무 많은 일을 함	◆ Extract Method (함수 추출) ◆ Introduce Parameter Object	Ctrl + Alt + M / ⌘ + ⌥ + M
②	거대한 클래스 (Large Class)	클래스가 너무 많은 책임을 가짐	◆ Extract Class (클래스 추출) ◆ Move Method	Ctrl + Alt + Shift + T → Extract Class / F6
③	중복 코드 (Duplicated Code)	동일 코드가 여러 곳에 반복	◆ Extract Method ◆ Template Method	Ctrl + Alt + M
④	Feature Envy (기능 질투)	한 클래스가 다른 클래스의 데이터를 과도하게 사용	◆ Move Method (메서드 이동)	F6
⑤	Data Clumps (데이터 덩어리)	여러 매개변수가 항상 함께 다님	◆ Introduce Parameter Object (매개변수 객체 도입)	Professional 전용 기능
⑥	Primitive Obsession (기초 타입 남용)	단순 타입(str, int 등)으로 복잡한 개념 표현	◆ Replace Data Value with Object	수동 작성 / Ctrl + Alt + Shift + T
⑦	Shotgun Surgery (산탄총 수술)	한 변경이 여러 파일에 영향을 미침	◆ Move Field ◆ Encapsulate Field	F6



5. 코드 스멜과 리팩토링 기법

대표적인 코드 스멜과 리팩토링 기법

구분	코드 스멜 (Code Smell)	문제점	리팩토링 기법	PyCharm 단축키
8	Divergent Change (뒤엉킨 변화)	한 클래스가 여러 이유로 자주 변경됨	◆ Extract Class (책임 분리)	Ctrl + Alt + Shift + T → Extract Class
9	Long Parameter List (매개변수 과다)	매개변수가 너무 많음	◆ Introduce Parameter Object ◆ Preserve Whole Object	Ctrl + F6
10	Lazy Class / Dead Code	거의 사용되지 않는 클래스나 메서드	◆ Inline Class ◆ Safe Delete	Alt + Delete / ⌘ + Delete
1 1	Temporary Field	특정 상황에서만 사용하는 필드 존재	◆ Extract Class	Ctrl + Alt + Shift + T
1 2	조건문 남용 (Switch Statements)	타입별로 if-elif문 반복	◆ Replace Conditional with Polymorphism	수동 리팩토링
1 3	이해하기 힘든 이름 (Mysterious Name)	함수/변수 이름이 모호함	◆ Rename (명확한 이름으로 변경)	Shift + F6
1 4	주석 남용 (Excessive Comments)	코드 자체가 불명확해서 주석이 많음	◆ Extract Method ◆ 명확한 네이밍	Ctrl + Alt + M / Shift + F6



6. 객체지향 설계의 5대 원칙

SOLID 원칙 개요

목적

- 유지보수성 향상
- 확장성 확보
- 테스트 용이성 증가

원칙	이름	핵심 개념
S	Single Responsibility	하나의 클래스는 하나의 책임만
O	Open/Closed	확장에는 열려있고 수정에는 닫혀있게
L	Liskov Substitution	상위 타입을 하위 타입으로 대체 가능
I	Interface Segregation	클라이언트 맞춤형 인터페이스
D	Dependency Inversion	추상화에 의존하라



6. 객체지향 설계의 5대 원칙

SOLID 원칙 개요

SOLID 5대 원칙 개요

원칙	영문	설명	핵심 질문
S	Single Responsibility	단일 책임	이 클래스가 변경되어야 하는 이유가 하나인가?
O	Open-Closed	개방-폐쇄	확장은 쉽고 수정은 불필요한가?
L	Liskov Substitution	리스코프 치환	부모를 자식으로 바꿔도 동작하는가?
I	Interface Segregation	인터페이스 분리	사용하지 않는 메서드에 의존하는가?
D	Dependency Inversion	의존성 역전	구체적인 것보다 추상적인 것에 의존하는가?



6. 객체지향 설계의 5대 원칙

최초의 코드

Guitar.py

```
class Guitar:
    def __init__(self, serial_number: str, price: float, maker: str, model: str,
                  top_wood: str, back_wood: str, string_num: int):
        self.serial_number = serial_number
        self.price = price
        self.maker = maker
        self.model = model
        self.top_wood = top_wood
        self.back_wood = back_wood
        self.string_num = string_num

    # getter methods
    def get_serial_number(self) -> str:
        return self.serial_number

    def get_price(self) -> float:
        return self.price

    def get_maker(self) -> str:
        return self.maker

    def get_model(self) -> str:
        return self.model

    def get_top_wood(self) -> str:
        return self.top_wood

    def get_back_wood(self) -> str:
        return self.back_wood
```

Guitar
+ serialNumber : String + price : Double + maker : Maker + model : String + topWood : Wood + backWood : Wood + stringNum : int
+ getSerialNumber() + getPrice() + getBuilder() + getType() + getBackWood() + getTopWood() + getStringNum() + getModel()



6. 객체지향 설계의 5대 원칙

최초의 코드

```
def get_string_num(self) -> int:  
    return self.string_num
```

```
def __str__(self):  
    return (f"Guitar(serial_number={self.serial_number}, maker={self.maker}, "  
            f"model={self.model}, price={self.price}, "  
            f"top_wood={self.top_wood}, back_wood={self.back_wood}, "  
            f"strings={self.string_num})")
```

```
if __name__ == "__main__":  
    my_guitar = Guitar(  
        serial_number="A12345",  
        price=1500.0,  
        maker="Fender",  
        model="Stratocaster",  
        top_wood="Maple",  
        back_wood="Alder",  
        string_num=6  
    )
```

```
print(my_guitar)  
print("Maker:", my_guitar.get_maker())  
print("Top wood:", my_guitar.get_top_wood())
```



6. 객체지향 설계의 5대 원칙

SRP, Single Responsibility Principle

단일 책임 원칙 (SRP, Single Responsibility Principle).

SRP는 클래스가 단 하나의 책임만 가져야 하며, 변경 사유가 하나뿐이어야 한다는 원칙

정의

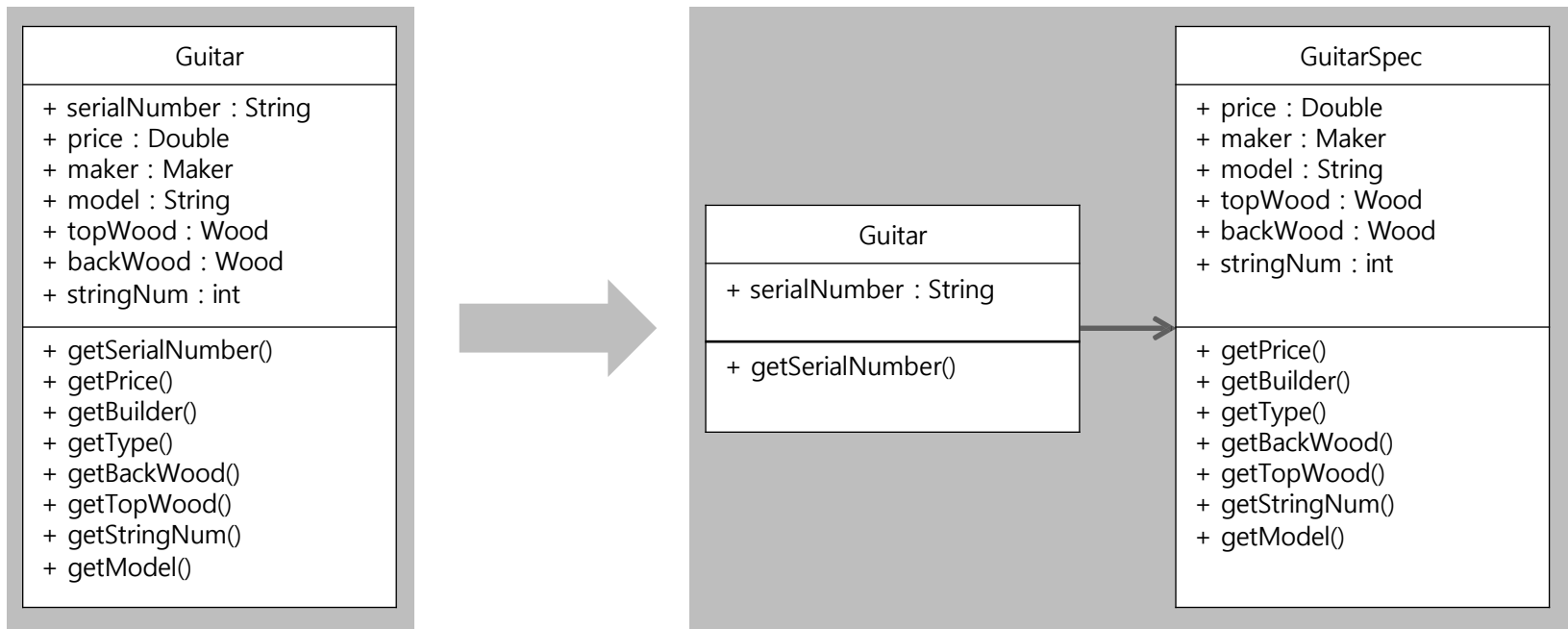
- 모든 클래스는 하나의 책임만 가지며, 클래스는 그 책임을 완전히 캡슐화해야 한다.
- 클래스가 제공하는 모든 기능은 이 책임과 부합해야 한다.
- 적용방법
- Extract Class를 통해 혼재된 각 책임을 각각의 개별 클래스로 분할하여 클래스 당 하나의 책임만을 맡도록 함
- 책임만 분리하는 것이 아니라 분리된 두 클래스간의 관계 복잡도를 줄이도록 설계
- Extract Class된 각각의 클래스들이 유사하고 비슷한 책임을 중복해서 갖고 있다면 Extract Superclass를 사용
- 책임을 기존의 어떤 클래스로 모으거나, 이렇만한 클래스가 없다면 새로운 클래스를 만들어 해결



6. 객체지향 설계의 5대 원칙

SRP, Single Responsibility Principle

SRP(Single Responsibility Principle, 단일 책임 원칙) 을 적용하여 Guitar 클래스의 책임을 두 개의 클래스로 분리(Extract Class)





6. 객체지향 설계의 5대 원칙



SRP, Single Responsibility Principle

GuitarSRP.py

```
# GuitarSpec 클래스: 기타의 세부 스펙 관리
class GuitarSpec:
    def __init__(self, price: float, maker: str, model: str,
                 top_wood: str, back_wood: str, string_num: int):
        self.price = price
        self.maker = maker
        self.model = model
        self.top_wood = top_wood
        self.back_wood = back_wood
        self.string_num = string_num

    # getter methods
    def get_price(self) -> float:
        return self.price

    def get_maker(self) -> str:
        return self.maker

    def get_model(self) -> str:
        return self.model

    def get_top_wood(self) -> str:
        return self.top_wood

    def get_back_wood(self) -> str:
        return self.back_wood

    def get_string_num(self) -> int:
        return self.string_num
```



6. 객체지향 설계의 5대 원칙

SRP, Single Responsibility Principle

```
def __str__(self):  
    return (f"GuitarSpec(maker={self.maker}, model={self.model}, "  
            f"price={self.price}, top_wood={self.top_wood}, "  
            f"back_wood={self.back_wood}, strings={self.string_num})")
```

Guitar 클래스: 기타의 고유 식별자 관리

class Guitar:

```
    def __init__(self, serial_number: str, spec: GuitarSpec):  
        self.serial_number = serial_number  
        self.spec = spec # GuitarSpec 객체 포함 (Composition)
```

```
    def get_serial_number(self) -> str:  
        return self.serial_number
```

```
    def get_spec(self) -> GuitarSpec:  
        return self.spec
```

```
    def __str__(self):  
        return f"Guitar(serial_number={self.serial_number}, spec={self.spec})"
```



6. 객체지향 설계의 5대 원칙

SRP, Single Responsibility Principle

```
# 사용 예시
if __name__ == "__main__":
    spec = GuitarSpec(
        price=1500.0,
        maker="Fender",
        model="Stratocaster",
        top_wood="Maple",
        back_wood="Alder",
        string_num=6
    )

    guitar = Guitar(serial_number="A12345", spec=spec)

    print("Guitar Info")
    print("Serial Number:", guitar.get_serial_number())
    print("Maker:", guitar.get_spec().get_maker())
    print("Model:", guitar.get_spec().get_model())
    print("Price:", guitar.get_spec().get_price())
```

```
C:\DEV\envs\py3_10_basic\Scrip
Guitar Info
Serial Number: A12345
Maker: Fender
Model: Stratocaster
Price: 1500.0
```

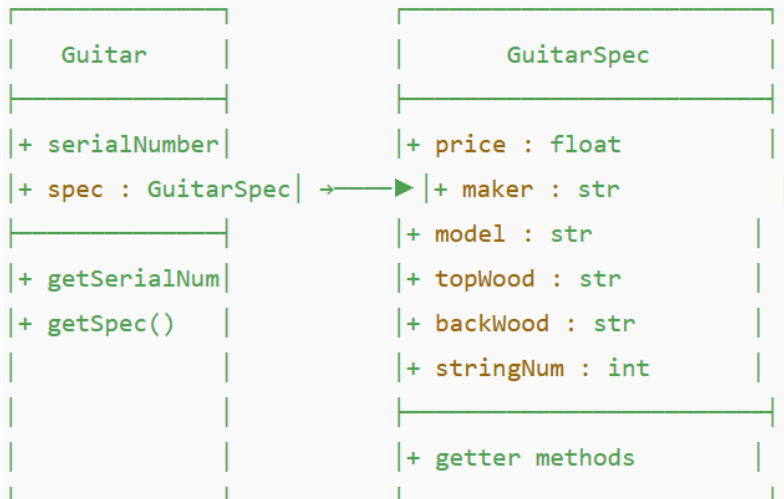


6. 객체지향 설계의 5대 원칙

SRP, Single Responsibility Principle

한 클래스가 너무 많은 일을 할 때, 책임을 나누어 응집도를 높인다.

- Guitar: 고유 식별 책임
- GuitarSpec: 속성 및 세부정보 관리 책임





6. 객체지향 설계의 5대 원칙

SRP, Single Responsibility Principle

하나의 클래스는 하나의 책임만 가져야 합니다.

✗ SRP 위반 (God Class)

```
class UserManager:
    def create_user(self, user_data):
        # DB에 사용자 저장
        pass

    def send_welcome_email(self, user_email):
        # 환영 이메일 발송
        pass

    def generate_user_report(self, user_id):
        # 사용자 리포트 생성
        pass

    def validate_password(self, password):
        # 비밀번호 검증
        pass
```



6. 객체지향 설계의 5대 원칙

SRP, Single Responsibility Principle

✓ SRP 준수 (책임 분리)

```
class UserRepository:
    """사용자 데이터 저장 책임"""
    def create(self, user_data):
        # DB에 사용자 저장
        pass

    def find_by_id(self, user_id):
        # 사용자 조회
        pass

class EmailService:
    """이메일 발송 책임"""
    def send_welcome_email(self, user_email):
        # 환영 이메일 발송
        pass

class UserReportGenerator:
    """리포트 생성 책임"""
    def generate_report(self, user_id):
        # 사용자 리포트 생성
        pass

class PasswordValidator:
    """비밀번호 검증 책임"""
    def validate(self, password):
        # 비밀번호 검증 (길이, 특수문자 등)
        pass
```

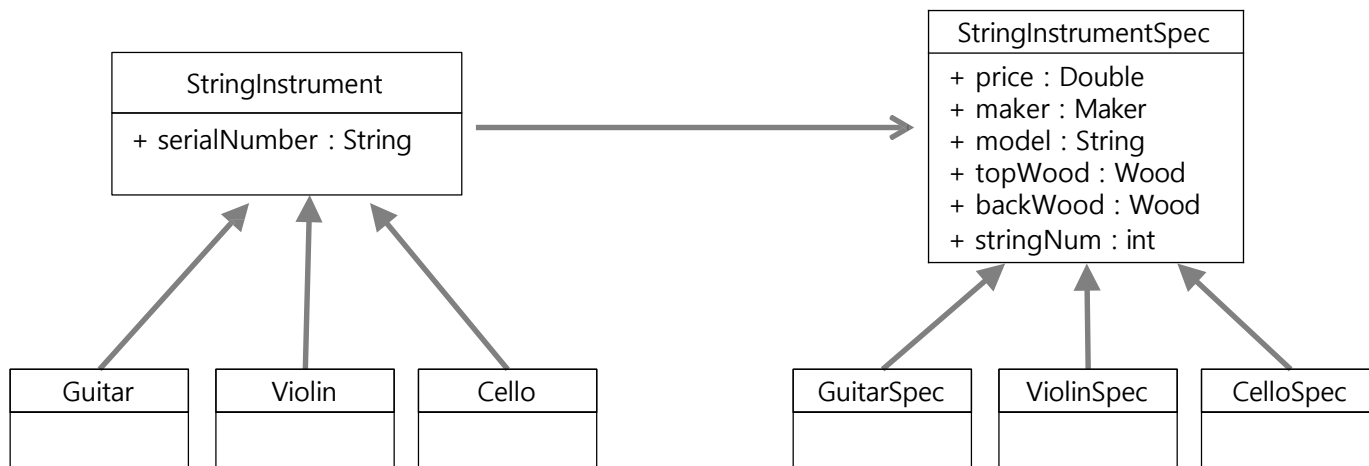


6. 객체지향 설계의 5대 원칙

OCP, Open/Closed Principle

만약 바이올린이나 첼로에 대한 정보가 필요하게 된다면?

개방-폐쇄 원칙(Open/Closed Principle, OCP)는 확장에는 열려(Open) 있고, 수정에는 닫혀(Closed) 있어야 한다는 객체지향 설계 원칙을 의미합니다.



OCP 준수 설계
새로운 악기나 스펙을 추가할 때 기존 클래스 (**StringInstrument**, **StringInstrumentSpec**)를 수정할 필요 없이 새로운 클래스만 확장하여 추가할 수 있습니다.
예를 들어:
첼로(**Cello**)를 추가하려면 **Cello**와 **CelloSpec** 클래스를 정의하면 됩니다. 기존 코드는 수정하지 않으므로, 시스템 안정성이 보장됩니다.



6. 객체지향 설계의 5대 원칙



OCP, Open/Closed Principle GuitarOCP.py

상위 클래스: 모든 현악기의 공통 속성

```
class StringInstrument:
    def __init__(self, serial_number: str, spec):
        self.serial_number = serial_number
        self.spec = spec # Composition (StringInstrumentSpec 또는 하위 클래스)

    def get_serial_number(self) -> str:
        return self.serial_number

    def get_spec(self):
        return self.spec
```

악기 스펙 상위 클래스

```
class StringInstrumentSpec:
    def __init__(self, price: float, maker: str, model: str,
                 top_wood: str, back_wood: str, string_num: int):
        self.price = price
        self.maker = maker
        self.model = model
        self.top_wood = top_wood
        self.back_wood = back_wood
        self.string_num = string_num

    def get_price(self) -> float:
        return self.price

    def get_maker(self) -> str:
        return self.maker
```



6. 객체지향 설계의 5대 원칙

OCP, Open/Closed Principle

```
def get_model(self) -> str:  
    return self.model
```

```
def get_top_wood(self) -> str:  
    return self.top_wood
```

```
def get_back_wood(self) -> str:  
    return self.back_wood
```

```
def get_string_num(self) -> int:  
    return self.string_num
```

```
def __str__(self):  
    return (f"{self.__class__.__name__}(maker={self.maker}, model={self.model}, "  
        f"price={self.price}, strings={self.string_num})")
```

개별 악기 스펙 (확장 가능)

```
class GuitarSpec(StringInstrumentSpec):  
    pass
```

```
class ViolinSpec(StringInstrumentSpec):  
    pass
```

```
class CelloSpec(StringInstrumentSpec):  
    pass
```



6. 객체지향 설계의 5대 원칙

OCP, Open/Closed Principle

```
# 구체 악기 클래스 (확장 가능)
class Guitar(StringInstrument):
    pass
```

```
class Violin(StringInstrument):
    pass
```

```
class Cello(StringInstrument):
    pass
```

```
# 사용 예시
if __name__ == "__main__":
    # 각 악기 스펙 정의
    guitar_spec = GuitarSpec(1500.0, "Fender", "Stratocaster", "Maple", "Alder", 6)
    violin_spec = ViolinSpec(4000.0, "Yamaha", "ClassicV", "Spruce", "Maple", 4)
    cello_spec = CelloSpec(6000.0, "Stradivarius", "SoloMaster", "Spruce", "Willow", 4)

    # 각 악기 생성
    guitar = Guitar("G123", guitar_spec)
    violin = Violin("V456", violin_spec)
    cello = Cello("C789", cello_spec)

    # 출력
    for instrument in [guitar, violin, cello]:
        print(f"{instrument.__class__.__name__}")
        print(f"Serial Number: {instrument.get_serial_number()}")
        print(f"Spec: {instrument.get_spec()}\n")
```

```
Guitar
Serial Number: G123
Spec: GuitarSpec(maker=Fender, model=Stratocaster, price=1500.0, strings=6)

Violin
Serial Number: V456
Spec: ViolinSpec(maker=Yamaha, model=ClassicV, price=4000.0, strings=4)

Cello
Serial Number: C789
Spec: CelloSpec(maker=Stradivarius, model=SoloMaster, price=6000.0, strings=4)
```



6. 객체지향 설계의 5대 원칙

| OCP, Open/Closed Principle

SRP (단일 책임 원칙)

- StringInstrument → “악기의 공통 정보(시리얼, 스펙)”만 담당
- StringInstrumentSpec → “스펙(재질, 가격 등)”만 담당
- 각 클래스가 한 가지 책임만 수행

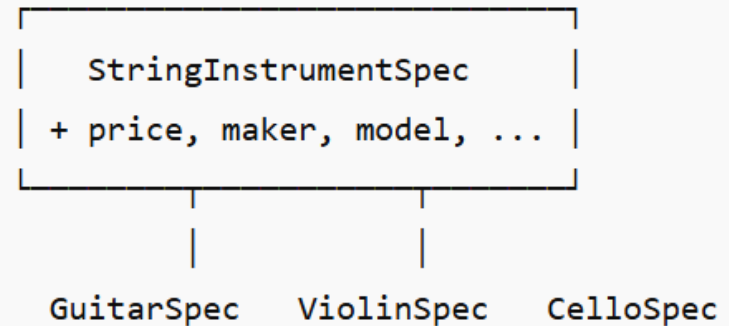
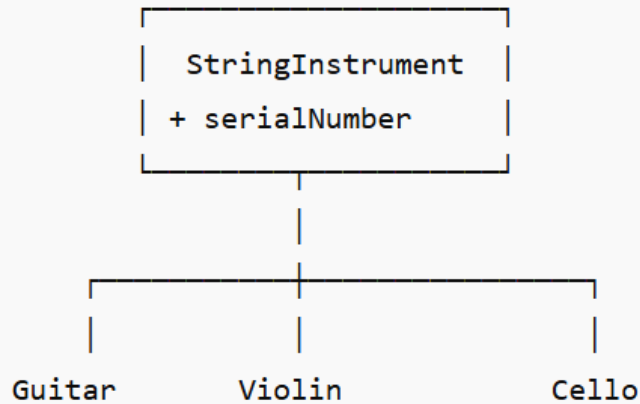
OCP (개방-폐쇄 원칙)

- 새로운 악기(예: Ukulele, Harp)를 추가해도 기존 코드 수정이 필요 없음. 확장은 가능(Open) 하지만 수정은 불필요(Closed)



6. 객체지향 설계의 5대 원칙

OCP, Open/Closed Principle





6. 객체지향 설계의 5대 원칙

| OCP, Open/Closed Principle

확장에는 열려 있고, 수정에는 닫혀 있어야 합니다.

✗ OCP 위반

```
class PaymentProcessor:
    def process_payment(self, amount, payment_type):
        if payment_type == "card":
            print(f"카드 결제: {amount}원")
        elif payment_type == "cash":
            print(f"현금 결제: {amount}원")
        elif payment_type == "kakaopay": # 새 결제 수단 추가 시 수정!
            print(f"카카오페이 결제: {amount}원")
```



6. 객체지향 설계의 5대 원칙

OCP, Open/Closed Principle



OCP 준수 (Strategy Pattern)

```
from abc import ABC, abstractmethod
```

```
class PaymentStrategy(ABC):
```

```
    @abstractmethod
```

```
    def pay(self, amount: int) -> None:
        pass
```

```
class CardPayment(PaymentStrategy):
```

```
    def pay(self, amount: int) -> None:
        print(f"카드 결제: {amount}원")
```

```
class CashPayment(PaymentStrategy):
```

```
    def pay(self, amount: int) -> None:
        print(f"현금 결제: {amount}원")
```

```
class KakaoPayment(PaymentStrategy): # 새 클래스 추가만!
```

```
    def pay(self, amount: int) -> None:
        print(f"카카오페이 결제: {amount}원")
```

```
class PaymentProcessor:
```

```
    def __init__(self, strategy: PaymentStrategy):
        self.strategy = strategy
```

```
    def process(self, amount: int) -> None:
        self.strategy.pay(amount)
```

```
# 사용
```

```
processor = PaymentProcessor(KakaoPayment())
```

```
processor.process(10000)
```



6. 객체지향 설계의 5대 원칙

Liskov Substitution Principle, LSP

리스코프 치환 원칙 (Liskov Substitution Principle, LSP)은 상위 클래스의 객체를 하위 클래스의 객체로 대체하더라도 프로그램의 동작에 문제가 없어야 한다는 원칙입니다.

정의

- 상위 타입의 객체를 하위 타입의 객체로 치환해도 상위 타입을 사용하는 프로그램은 정상적으로 동작되어야 함 (하위는 상위 타입이 약속한 규약을 지켜야 함)
- 다형성에 관한 원칙으로 개방/폐쇄 원칙을 지원하기 위한 원칙

위반사례

- 명시된 명세에서 벗어난 Exception을 발생시키는 경우, 명시된 명세에서 벗어난 기능을 수행 (downcasting 발생)
- Animal 인터페이스가 makeNoise() 메소드를 갖는다면, 하위 클래스들을 적절히 구현해야 함 (고양이는 야옹, 강아지는 멍멍 등)
- DontActuallyMakeNoiseException을 발생시키는 MuteMouse와 같은 하위 클래스를 만들지 말아야 함
- 이 객체는 Animal를 비즈니스적으로 구현한 것이 아니다

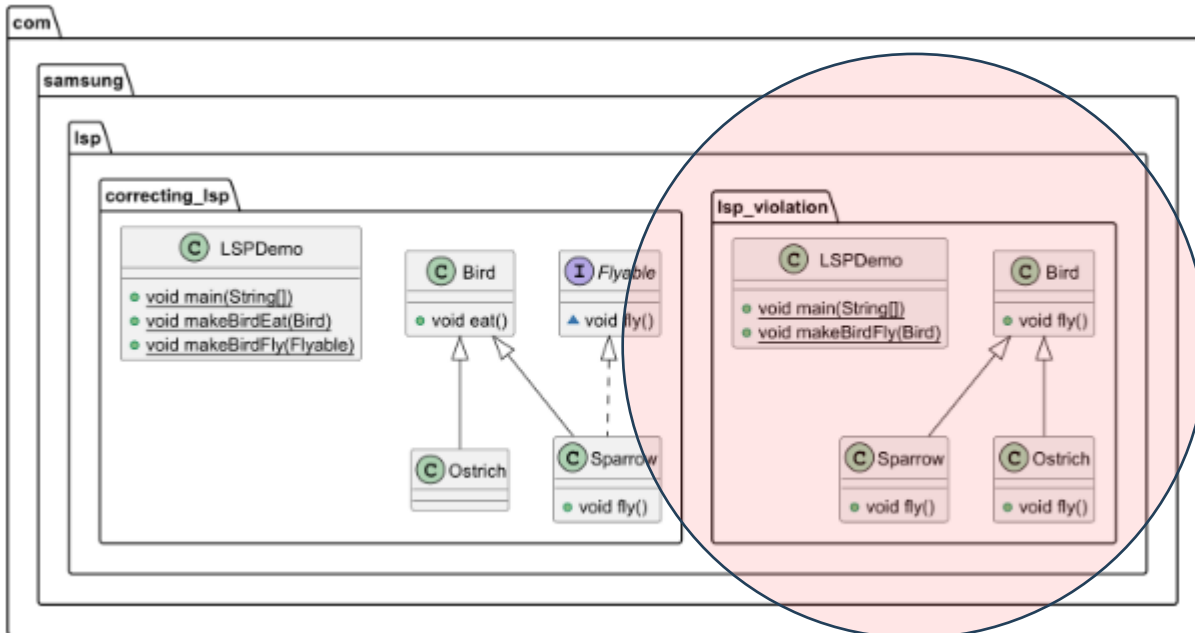


6. 객체지향 설계의 5대 원칙

Liskov Substitution Principle, LSP

Example :

- 잘못된 예: LSP 위반
- 문제점: LSP 위반
 - Bird 클래스는 모든 새가 날 수 있다는 가정을 가지고 있지만, Ostrich(타조)는 날 수 없습니다.
 - 결과적으로 Ostrich 객체를 Bird 타입으로 치환할 경우 프로그램이 제대로 동작하지 않습니다.



상위 클래스의 객체를 하위 클래스의 객체로 대체하더라도 프로그램의 동작에 문제가 없어야 함.



6. 객체지향 설계의 5대 원칙

Liskov Substitution Principle, LSP

LSP_X.py

```
class Bird:
    def fly(self):
        print("새가 날고 있습니다.")
```

```
class Sparrow(Bird):
    def fly(self):
        print("참새가 날고 있습니다.")
```

```
class Ostrich(Bird):
    def fly(self):
        # ✗ 타조는 날 수 없는데, 상속받은 fly() 메서드로 인해 논리적 오류 발생
        raise Exception("타조는 날 수 없습니다!")
```

```
def make_bird_fly(bird: Bird):
    bird.fly()
```

```
if __name__ == "__main__":
    sparrow = Sparrow()
    ostrich = Ostrich()
```

```
make_bird_fly(sparrow) # ✅ 정상 작동
make_bird_fly(ostrich) # ✗ LSP 위반 - 예외 발생
```

```
Traceback (most recent call last):
  File "C:\DEV\SamsungDisplay\Refactoring_practice\LSP_X.py", line 26, in <module>
    make_bird_fly(ostrich) # ✗ LSP 위반 - 예외 발생
  File "C:\DEV\SamsungDisplay\Refactoring_practice\LSP_X.py", line 18, in make_bird_fly
    bird.fly()
  File "C:\DEV\SamsungDisplay\Refactoring_practice\LSP_X.py", line 14, in fly
    raise Exception("타조는 날 수 없습니다!")
Exception: 타조는 날 수 없습니다!
```



6. 객체지향 설계의 5대 원칙

Liskov Substitution Principle, LSP

Example :

수정 내용

1. Flyable 인터페이스:

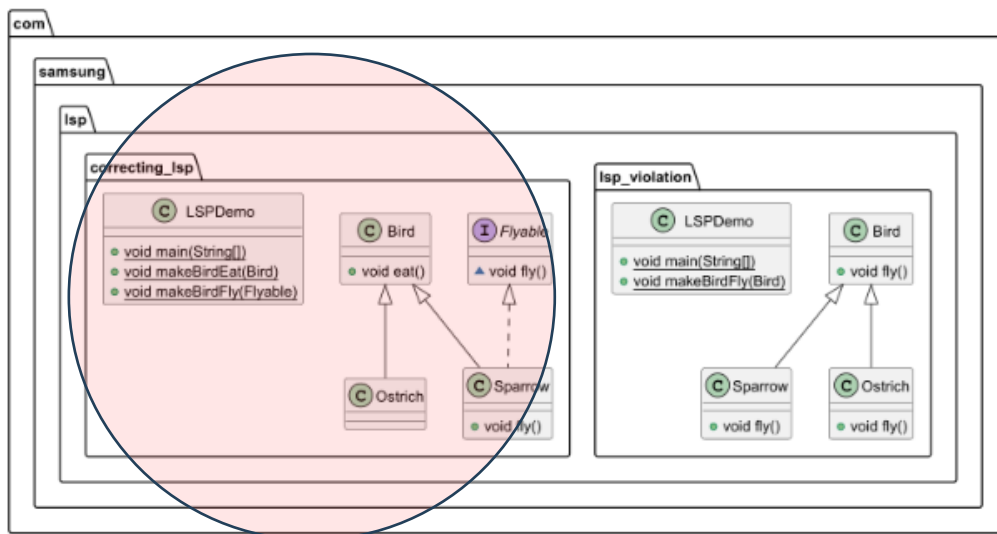
- 날 수 있는 새만 Flyable 인터페이스를 구현하도록 설계합니다.
- Sparrow는 Flyable을 구현하지만, Ostrich는 이를 구현하지 않습니다.

2. 행동 분리:

- 모든 새는 eat() 메서드를 가질 수 있지만, fly() 메서드는 날 수 있는 새만 가집니다.
- Bird 클래스에서 fly()를 분리하여 LSP를 준수하도록 만듭니다.

3. LSP 준수:

- Bird 타입의 객체는 항상 안전하게 치환될 수 있습니다.
- Flyable 인터페이스는 날 수 있는 새의 특화된 행동만 포함합니다.





6. 객체지향 설계의 5대 원칙



Liskov Substitution Principle, LSP

LSP_O.py

```
from abc import ABC, abstractmethod
```

```
# ❶ 공통 상위 클래스: 모든 새의 기본 행동 정의
class Bird(ABC):
    def eat(self):
        print("새가 먹고 있습니다.")
```

```
# ❷ 날 수 있는 새만 구현할 인터페이스 (추상 클래스)
class Flyable(ABC):
    @abstractmethod
    def fly(self):
        pass
```

```
# ❸ 실제 새 클래스들
class Sparrow(Bird, Flyable):
    def fly(self):
        print("참새가 날고 있습니다.")
```

```
class Ostrich(Bird): # ✗ Flyable을 상속하지 않음
    def run(self):
        print("타조가 달리고 있습니다.")
```



6. 객체지향 설계의 5대 원칙



Liskov Substitution Principle, LSP

4 LSP 준수 테스트 함수

```
def make_bird_move(bird: Bird):  
    bird.eat() # 모든 새가 공통으로 먹을 수 있음  
  
    if isinstance(bird, Flyable):  
        bird.fly() # 날 수 있는 새만 fly() 호출  
    else:  
        print("이 새는 날지 않습니다.")
```

5 실행 예시

```
if __name__ == "__main__":  
    sparrow = Sparrow()  
    ostrich = Ostrich()  
  
    print("🐦 [참새 행동]")  
    make_bird_move(sparrow)  
  
    print("\n🦩 [타조 행동]")  
    make_bird_move(ostrich)
```



6. 객체지향 설계의 5대 원칙

Liskov Substitution Principle, LSP

모든 Bird 객체(Sparrow, Ostrich)를 Bird 타입으로 대체해도 동작에 문제가 없어야 함. Flyable 인터페이스로 행동을 명확히 분리

- Bird는 모든 새의 공통 행동(eat)만 정의
- Ostrich는 날지 않는 새라 정의하되, 프로그램의 동작에는 문제 없음



6. 객체지향 설계의 5대 원칙

Liskov Substitution Principle, LSP

부모 클래스를 자식 클래스로 치환해도 정상 동작해야 합니다.

✗ LSP 위반

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def set_width(self, width):
        self.width = width

    def set_height(self, height):
        self.height = height

    def get_area(self):
        return self.width * self.height

class Square(Rectangle): # 정사각형은 직사각형?
    def set_width(self, width):
        self.width = width
        self.height = width # 높이도 함께 변경!

    def set_height(self, height):
        self.width = height # 너비도 함께 변경!
        self.height = height

# 문제 발생
def test_rectangle(rect: Rectangle):
    rect.set_width(5)
    rect.set_height(4)
    assert rect.get_area() == 20 # 직사각형은 통과
    # Square는 실패! (area = 16)
```



6. 객체지향 설계의 5대 원칙

Liskov Substitution Principle, LSP



LSP 준수

```
from abc import ABC, abstractmethod
```

```
class Shape(ABC):  
    @abstractmethod  
    def get_area(self) -> float:  
        pass
```

```
class Rectangle(Shape):  
    def __init__(self, width: float, height: float):  
        self.width = width  
        self.height = height
```

```
    def get_area(self) -> float:  
        return self.width * self.height
```

```
class Square(Shape):  
    def __init__(self, side: float):  
        self.side = side
```

```
    def get_area(self) -> float:  
        return self.side * self.side
```




6. 객체지향 설계의 5대 원칙

Interface Segregation Principle, ISP

인터페이스 분리 원칙 (Interface Segregation Principle, ISP)은 클라이언트가 자신이 사용하지 않는 메서드에 의존하지 않도록 인터페이스를 분리해야 한다는 원칙입니다. 즉, 인터페이스는 특정 클라이언트의 요구에 맞게 작고 구체적으로 설계되어야 합니다.

정의

- 어떤 클래스가 다른 클래스를 사용할 때에 가능한 최소한의 인터페이스만을 사용해야 한다.
- '하나의 일반적인 인터페이스보다, 여러 개의 구체적인 인터페이스가 낫다'
- SRP: 클래스에 대한 단일책임, ISP: 인터페이스의 단일책임

적용사례

- Swing의 JTable의 굉장히 많은 메소드를 갖으나, 특정 역할에 대한 인터페이스를 통해 기능을 제공 (Accessible, CellEditorListener, ListSelectionListener, Scrollable, TableModelListener, TableColumnModelListener 등)



6. 객체지향 설계의 5대 원칙

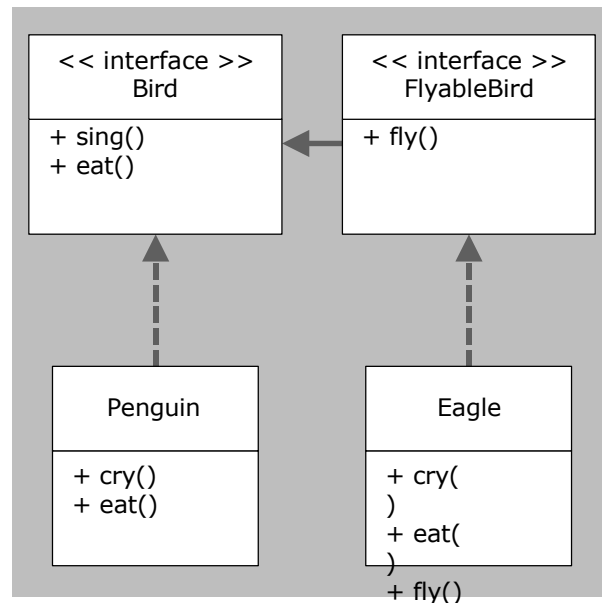
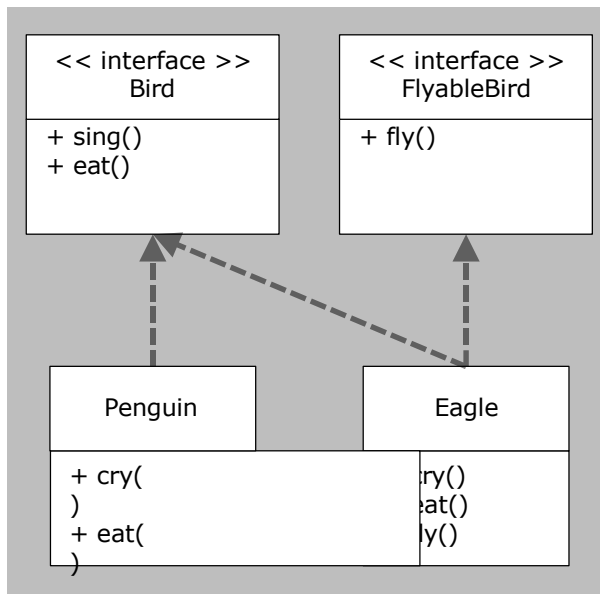
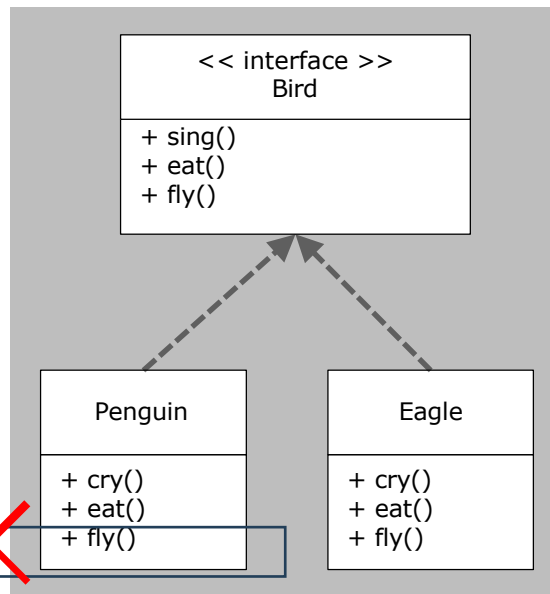
Interface Segregation Principle, ISP

적용방법

기 구현된 클라이언트에 변경을 주지 말아야 한다.

두 개 이상의 인터페이스가 공유하는 부분의 재사용을 극대화한다.

서로 다른 성격의 인터페이스를 명백히 분리한다.





6. 객체지향 설계의 5대 원칙

Interface Segregation Principle, ISP

인터페이스 분리:

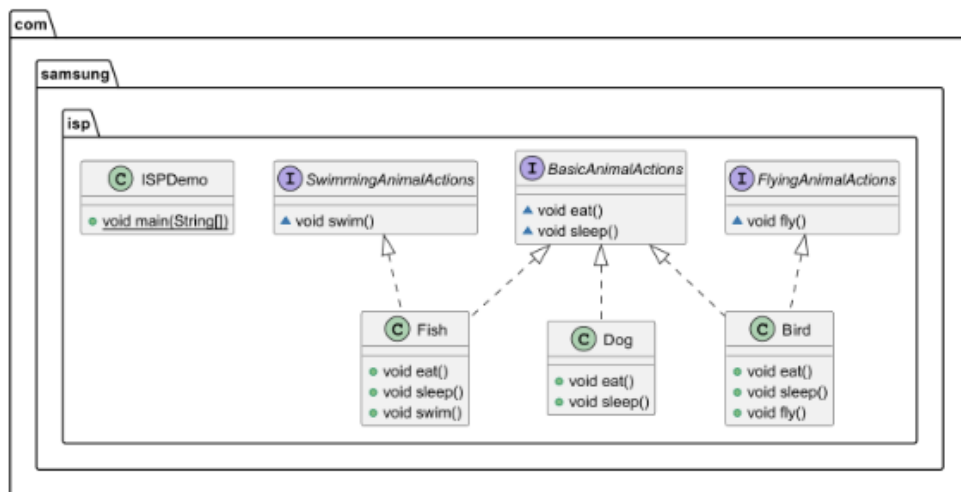
- BasicAnimalActions: 모든 동물이 공통으로 가지는 행동 (eat, sleep).
- FlyingAnimalActions: 날 수 있는 동물을 위한 행동 (fly).
- SwimmingAnimalActions: 수영할 수 있는 동물을 위한 행동 (swim).

구현 클래스:

- Bird: 기본 행동과 날기 기능을 모두 구현.
- Fish: 기본 행동과 수영 기능을 모두 구현.
- Dog: 기본 행동만 구현.

ISP 준수:

- 각 클래스는 자신과 관련 없는 메서드를 강제로 구현하지 않습니다.
- 예를 들어, Dog는 fly()나 swim() 같은 메서드를 가지지 않습니다.





6. 객체지향 설계의 5대 원칙



Interface Segregation Principle, ISP

ISP.py

```
from abc import ABC, abstractmethod
```

```
# 1. 기본 인터페이스: 모든 동물이 공통적으로 수행하는 행동
```

```
class BasicAnimalActions(ABC):
```

```
    @abstractmethod
```

```
    def eat(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def sleep(self):
```

```
        pass
```

```
# 2. 수영 가능한 동물 인터페이스
```

```
class SwimmingAnimalActions(ABC):
```

```
    @abstractmethod
```

```
    def swim(self):
```

```
        pass
```

```
# 3. 날 수 있는 동물 인터페이스
```

```
class FlyingAnimalActions(ABC):
```

```
    @abstractmethod
```

```
    def fly(self):
```

```
        pass
```



6. 객체지향 설계의 5대 원칙

Interface Segregation Principle, ISP

```
# Fish: 수영할 수 있는 동물
class Fish(BasicAnimalActions, SwimmingAnimalActions):
    def eat(self):
        print("물고기가 먹고 있습니다.")
```

```
    def sleep(self):
        print("물고기가 잠자고 있습니다.")
```

```
    def swim(self):
        print("물고기가 헤엄치고 있습니다.")
```

```
# Dog: 수영하지 않지만 기본 행동만 수행
class Dog(BasicAnimalActions):
    def eat(self):
        print("개가 밥을 먹고 있습니다.")
```

```
    def sleep(self):
        print("개가 잠을 잡니다.")
```

```
# Bird: 날 수 있는 동물
class Bird(BasicAnimalActions, FlyingAnimalActions):
    def eat(self):
        print("새가 벌레를 먹고 있습니다.")
```

```
    def sleep(self):
        print("새가 나뭇가지 위에서 잠듭니다.")
```

```
    def fly(self):
        print("새가 하늘을 날고 있습니다.")
```



6. 객체지향 설계의 5대 원칙

Interface Segregation Principle, ISP

ISP 원칙 데모 (ISP Demo)

```
def main():
```

```
    animals = [Fish(), Dog(), Bird()]
```

```
    for animal in animals:
```

```
        print(f"\n[{animal.__class__.__name__}]")
```

```
        animal.eat()
```

```
        animal.sleep()
```

```
    if isinstance(animal, SwimmingAnimalActions):
```

```
        animal.swim()
```

```
    if isinstance(animal, FlyingAnimalActions):
```

```
        animal.fly()
```

```
if __name__ == "__main__":
```

```
    main()
```

[Fish]

물고기가 먹고 있습니다.

물고기가 잠자고 있습니다.

물고기가 헤엄치고 있습니다.

[Dog]

개가 밥을 먹고 있습니다.

개가 잠을 잡니다.

[Bird]

새가 벌레를 먹고 있습니다.

새가 나뭇가지 위에서 잠듭니다.

새가 하늘을 날고 있습니다.



6. 객체지향 설계의 5대 원칙

Interface Segregation Principle, ISP

- Dog는 날지도, 수영하지도 않지만 eat()과 sleep()만 구현.
- Fish는 수영만 구현.
- Bird는 비행만 구현.

이로써 각 클래스는 “자신에게 필요한 기능만” 의존하게 되어 ISP(인터페이스 분리 원칙) 을 완벽히 준수 Dog는 날지도, 수영하지도 않지만 eat()과 sleep()만 구현.

- Fish는 수영만 구현.
- Bird는 비행만 구현.

이로써 각 클래스는 “자신에게 필요한 기능만” 의존하게 되어 ISP(인터페이스 분리 원칙) 을 완벽히 준수



6. 객체지향 설계의 5대 원칙

Interface Segregation Principle, ISP

클라이언트는 자신이 사용하지 않는 인터페이스에 의존하지 않아야 합니다.

✗ ISP 위반 (Fat Interface)

```
from abc import ABC, abstractmethod
```

```
class Worker(ABC):
```

```
    @abstractmethod
```

```
    def work(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def eat(self):
```

```
        pass
```

```
    @abstractmethod
```

```
    def sleep(self):
```

```
        pass
```

```
class Human(Worker):
```

```
    def work(self):
```

```
        print("일하기")
```

```
    def eat(self):
```

```
        print("먹기")
```

```
    def sleep(self):
```

```
        print("자기")
```

```
class Robot(Worker): # 로봇은 먹고 자지 않는데...
```

```
    def work(self):
```

```
        print("일하기")
```

```
    def eat(self):
```

```
        pass # 불필요한 메서드 구현!
```

```
    def sleep(self):
```

```
        pass # 불필요한 메서드 구현!
```




6. 객체지향 설계의 5대 원칙



Interface Segregation Principle, ISP



ISP 준수 (Interface Segregation)

```
from abc import ABC, abstractmethod
```

```
class Workable(ABC):  
    @abstractmethod  
    def work(self):  
        pass
```

```
class Eatable(ABC):  
    @abstractmethod  
    def eat(self):  
        pass
```

```
class Sleepable(ABC):  
    @abstractmethod  
    def sleep(self):  
        pass
```

```
class Human(Workable, Eatable, Sleepable):  
    def work(self):  
        print("일하기")
```

```
    def eat(self):  
        print("먹기")
```

```
    def sleep(self):  
        print("자기")
```

```
class Robot(Workable): # 필요한 인터페이스만!  
    def work(self):  
        print("일하기")
```



6. 객체지향 설계의 5대 원칙

Dependency Inversion Principle, DIP

의존성 역전 원칙 (Dependency Inversion Principle, DIP)은 다음을 원칙으로 합니다.

- 상위 모듈(고수준 모듈)은 하위 모듈(저수준 모듈)에 의존해서는 안 된다. 둘 다 추상화(인터페이스)에 의존해야 한다.
- 추상화는 세부사항에 의존하지 않아야 한다. 세부사항이 추상화에 의존해야 한다.

정의

- 소프트웨어 모듈 분리의 특정 형식으로 상위 계층이 하위 세부 계층에 의존하는 전통적인 의존관계를 역전시킴으로써 상위 계층이 하위 계층의 구현으로부터 독립되게 할 수 있다.
- 상위 모듈은 하위 모듈에 의존해서는 안됨 (상위/하위 모듈 모두 추상화에 의존해야 함)
- 추상화는 세부 사항에 의존해서는 안됨 (세부사항이 추상화에 의존해야 함)



6. 객체지향 설계의 5대 원칙

Dependency Inversion Principle, DIP

추상화 (MessageService):

- 상위 모듈(Notification)이 의존하는 공통 인터페이스입니다.
- 고수준 모듈과 저수준 모듈 간의 결합도를 낮춥니다.

구체적인 구현 (EmailService, SMSService):

- 각각 MessageService 인터페이스를 구현하여 이메일과 SMS 전송 기능을 제공합니다.
- Notification 클래스는 구현체를 직접 알 필요가 없습니다.

상위 모듈 (Notification):

- 인터페이스(MessageService)에 의존하기 때문에 이메일, SMS, 또는 다른 메시징 서비스와 같은 구체적인 구현에 의존하지 않습니다.

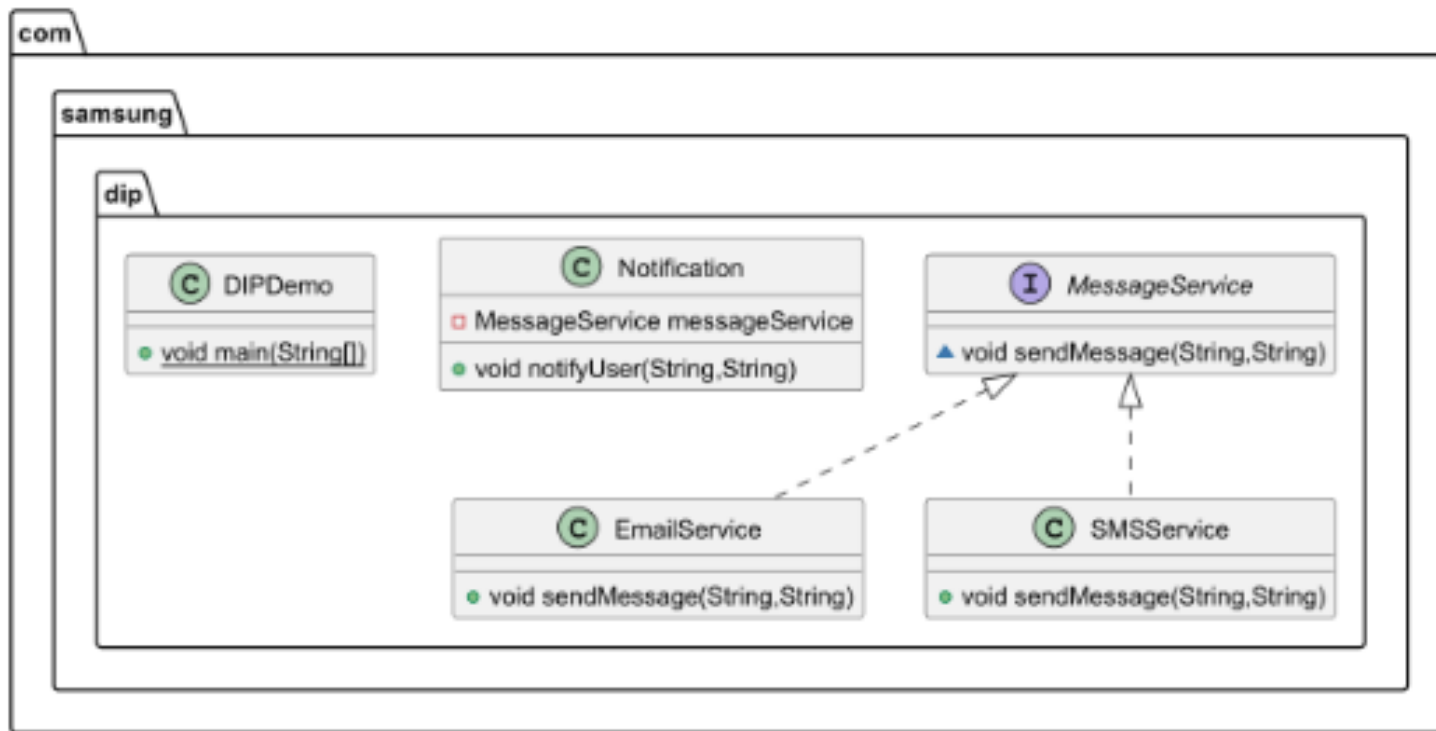
의존성 주입:

- Notification 클래스는 생성자를 통해 MessageService를 주입받습니다. 이는 DIP를 준수하며 확장성을 높입니다.



6. 객체지향 설계의 5대 원칙

Dependency Inversion Principle, DIP





6. 객체지향 설계의 5대 원칙



Dependency Inversion Principle, DIP DIP.py

```
from abc import ABC, abstractmethod
```

```
# 상위 추상화 계층 (Interface)
```

```
class MessageService(ABC):
```

```
    @abstractmethod
```

```
    def send_message(self, recipient: str, content: str):  
        pass
```

```
# 구체적 구현 1: EmailService
```

```
class EmailService(MessageService):
```

```
    def send_message(self, recipient: str, content: str):
```

```
        print(f"[이메일 전송] 받는 사람: {recipient}, 내용: {content}")
```

```
# 구체적 구현 2: SMSService
```

```
class SMSService(MessageService):
```

```
    def send_message(self, recipient: str, content: str):
```

```
        print(f"[SMS 전송] 받는 사람: {recipient}, 내용: {content}")
```

```
# 상위 모듈: Notification — 구체 클래스가 아닌 추상 인터페이스에 의존함
```

```
class Notification:
```

```
    def __init__(self, message_service: MessageService):
```

```
        self.message_service = message_service # 추상화에 의존
```

```
    def notify_user(self, user: str, message: str):
```

```
        self.message_service.send_message(user, message)
```



6. 객체지향 설계의 5대 원칙

Dependency Inversion Principle, DIP

```
# DIP Demo
def main():
    email_service = EmailService()
    sms_service = SMSService()

    # Email 기반 알림
    email_notifier = Notification(email_service)
    email_notifier.notify_user("alice@example.com", "이메일 알림이 도착했습니다!")

    # SMS 기반 알림
    sms_notifier = Notification(sms_service)
    sms_notifier.notify_user("010-1234-5678", "문자 알림이 도착했습니다!")

if __name__ == "__main__":
    main()
```

```
C:\DEV\envs\py3_10_basic\Scripts\python.exe C:\DEV\SamsungDisplay\Refactori
```

```
[이메일 전송] 받는 사람: alice@example.com, 내용: 이메일 알림이 도착했습니다!
```

```
[SMS 전송] 받는 사람: 010-1234-5678, 내용: 문자 알림이 도착했습니다!
```

```
Process finished with exit code 0
```



6. 객체지향 설계의 5대 원칙

Dependency Inversion Principle, DIP

“상위 모듈이 하위 모듈의 세부 사항에 의존하지 않고,
하위 모듈이 상위 모듈이 정의한 추상화(인터페이스)에 의존해야 한다.”

구분	잘못된 설계	개선된 설계 (DIP 적용)
의존 관계	Notification → EmailService 직접 의존	Notification → MessageService 추상화 의존
결합도	높음 (하나의 구현에 종속)	낮음 (언제든 교체 가능)
유연성	확장 어려움	SlackService, PushService 등 손쉽게 추가 가능



6. 객체지향 설계의 5대 원칙

Dependency Inversion Principle, DIP

구체적인 것이 아닌 추상적인 것에 의존해야 합니다.

✗ DIP 위반

```
class MySQLDatabase: # 구체 클래스
    def save(self, data):
        print(f"MySQL에 저장: {data}")
```

```
class UserService:
    def __init__(self):
        self.db = MySQLDatabase() # 구체 클래스에 직접 의존!

    def create_user(self, user_data):
        self.db.save(user_data)
```




6. 객체지향 설계의 5대 원칙

Dependency Inversion Principle, DIP



DIP 준수

```
from abc import ABC, abstractmethod

class Database(ABC): # 추상 인터페이스
    @abstractmethod
    def save(self, data):
        pass

class MySQLDatabase(Database):
    def save(self, data):
        print(f"MySQL에 저장: {data}")

class PostgreSQLDatabase(Database):
    def save(self, data):
        print(f"PostgreSQL에 저장: {data}")

class UserService:
    def __init__(self, db: Database): # 추상화에 의존!
        self.db = db

    def create_user(self, user_data):
        self.db.save(user_data)

# 사용 (의존성 주입)
mysql_db = MySQLDatabase()
user_service = UserService(mysql_db)

# DB 교체 쉬움
postgres_db = PostgreSQLDatabase()
user_service = UserService(postgres_db)
```

7. 실습

| 실습1 : Clean Code + UnitTest + 리팩토링
다음 코드를 리팩토링 합니다.

```
# ❌ 나쁜 코드 (Code Smell)
def p(u, m, q): # 의미 없는 변수명
    if u and m:
        t = m.price * q
        if t > 10000: # 매직 넘버
            t = t * 0.9
        return t
    return 0
```

『4과목』

리팩토링 + 코드 리뷰 + 배포

- Part 1. 오리엔테이션 및 목표
- Part 2. 리팩토링 이론
- Part 3. 코드 리뷰 + 배포
- Part 3. 종합 실습 & 마무리





1. Code Review & Static Analysis

코드 리뷰(Code Review)란?

사람(개발자)이 직접 코드를 보고
기능, 설계, 가독성, 유지보수성 등을 검토하는 과정입니다.

✓ 검토 대상 예시

- 로직이 비즈니스 요구사항에 맞는가?
- 테스트가 충분한가?
- 명확한 변수명과 구조인가?
- 성능/확장성 문제는 없는가?

→ 인간 판단이 필요한 부분을 보완



1. Code Review & Static Analysis

Python 정적 분석(Static Analysis)이란?

정적 분석은 코드를 실행하지 않고
코드의 문법, 스타일, 보안, 품질 문제를 자동으로 검사하는 기술.

✓ 발견 가능한 문제들

- 문법 오류
- 잠재적 버그
- 스타일/PEP8 위반
- 사용되지 않는 코드
- 보안 취약점
- 복잡도/유지보수 문제

→ 목적: 초기에 결함을 발견하고 비용을 줄이는 것



1. Code Review & Static Analysis

Python 정적 분석(Static Analysis)이란?

정적 분석이 중요한 이유

장점	기대 효과
문제 조기 발견	런타임 오류 예방, 비용 절감
코드 일관성 확보	가독성 향상, 협업 생산성 증가
보안 취약점 예방	금융/의료 등 고위험 환경에서 필수
복잡도 관리	유지보수 쉬운 코드 구조
CI 자동화 가능	품질 관리 체계화



1. Code Review & Static Analysis

정적 분석 vs 코드 리뷰 비교

가장 좋은 방식: 두 방법을 함께 적용

항목	정적 분석	코드 리뷰
수행 주체	자동 시스템	사람
스타일/포맷 검증	뛰어남	가능
보안/버그 탐지	자동화 가능	가능
비즈니스 로직 검증	불가능	매우 중요
설계·구조적 개선	제한적	핵심 역할



1. Code Review & Static Analysis



실전에서 잡아주는 문제 예시

정적 분석 도구

- 미사용 변수
- 오타로 인한 예외 가능성
- 보안 취약한 코드
- 복잡도 과다 함수



1. Code Review & Static Analysis



실전에서 잡아주는 문제 예시

사람 코드 리뷰

- 비즈니스 로직 잘못 적용
- 가독성/의도 불분명한 코드
- 중복 로직
- 확장성 문제



1. Code Review & Static Analysis

Static Analysis?

Static Analysis = 코드를 실행하지 않고 정적(Static) 으로 코드 품질, 오류, 스타일을 검사하는 것.

Python에서는 크게 4가지 범주로 나눌 수 있습니다:

1. Linting Tools (스타일 검사 + 잠재적 오류 탐지)
2. Type Checking Tools (정적 타입 분석)
3. Security Scanning Tools (보안 취약점 분석)
4. Code Quality / Complexity Tools (코드 복잡도 분석)



1. Code Review & Static Analysis

1. Linting Tools (Lint = 스타일 + 오류 검사)

1) Pylint

- 가장 엄격한 스타일 + 오류 분석기
- 변수명, 함수명, 코드 smells, import 문제까지 탐지

설치

```
pip install pylint
```

실행

```
pylint your_file.py
```



1. Code Review & Static Analysis

1. Linting Tools (Lint = 스타일 + 오류

2) Flake8

- 가장 널리 사용되는 Python linting tool
- 빠르고 가벼움
- PEP8 스타일 검사 + 기본 오류 탐지

설치

```
pip install flake8
```

실행

```
flake8 your_file.py
```



1. Code Review & Static Analysis

1. Linting Tools (Lint = 스타일 + 오류 검사)

3) Ruff (최신, 매우 빠른 정적 분석기)

- Rust 기반 → Flake8보다 100배 빠름
- Linting + 포맷터(Black) 역할 일부도 수행

설치

```
pip install ruff
```

실행

```
ruff check .
```



1. Code Review & Static Analysis

2. Type Checking Tools (정적 타입 분석)

1) MyPy

- Python 타입 힌트를 분석하여 타입 오류를 탐지함

설치

```
pip install mypy
```

실행

```
mypy your_file.py
```



1. Code Review & Static Analysis

2. Type Checking Tools (정적 타입 분석)

2) Pylint (Microsoft – VSCode 내장)

- 매우 빠름
- TypeScript 팀이 만든 정적 타입 분석기

설치

```
npm install -g pylint
```

실행

```
pylint
```



1. Code Review & Static Analysis

2. Type Checking Tools (정적 타입 분석)

3) Pylance

- Pyright 기반의 VS Code 확장
- 자동 타입 오류 표시

설치: VSCode → Extensions → Pylance



1. Code Review & Static Analysis

3. Security Scanning Tools (보안 취약점 분석)

1) Bandit

- Python 코드 내 보안 위험 코드 탐지
- 암호화 문제, eval 사용, 하드코딩된 비밀번호 등 탐지

설치

```
pip install bandit
```

실행

```
bandit -r .
```



1. Code Review & Static Analysis

3. Security Scanning Tools (보안 취약점 분석)

2) Safety

- 설치된 Python 패키지에서 취약점이 있는 버전을 탐지

설치

```
pip install safety
```

실행

```
safety check
```



1. Code Review & Static Analysis

3. Security Scanning Tools (보안 취약점 분석)

3) Semgrep

- 언어 전반을 지원하는 고급 보안 규칙 기반 스캐너
- DevSecOps CI에서 많이 사용

설치

```
pip install semgrep
```

실행

```
semgrep --config=auto .
```



1. Code Review & Static Analysis

4. Code Quality & Complexity Tools

1) Radon

- 코드 복잡도 분석 (Cyclomatic complexity)
- 유지보수 하기 어려운 코드를 찾아줌

설치

```
pip install radon
```

복잡도 분석

```
radon cc -s -a .
```

Maintainability Index

```
radon mi -s your_file.py
```



1. Code Review & Static Analysis

4. Code Quality & Complexity Tools

2) SonarQube / SonarCloud

- 매우 강력한 기업용 정적 분석 플랫폼
- 코드 스멜, 보안, 테스트 커버리지까지 통합

실행 예

CI/CD에서 SonarScanner 사용

sonar-scanner



1. Code Review & Static Analysis

5. Formatting Tools (자동 코드 정리 - 정적 분석과 함께 사용)

black .

isort .

ruff check .

도구	역할
Black	코드 자동 포매터
isort	import 순서를 자동 정렬
Ruff	포매터 + 린터 통합



1. Code Review & Static Analysis

팀에서의 실제 적용 워크플로우

CI/CD 자동화와 결합 예:

1. 개발자가 코드 push →
2. 정적 분석 자동 실행 →
3. 문제 해결 →
4. Pull Request 생성 →
5. 사람이 코드 리뷰 →
6. Merge & Deploy



1. Code Review & Static Analysis



팀에서의 실제 적용 워크플로우

GitHub Actions 예시:

- run: flake8 .
- run: black --check .
- run: mypy .
- run: bandit -r .



1. Code Review & Static Analysis

실습2 : 정적 분석 도구 flake8과 pylint

(py3_10_basic)

C:\DEV\SamsungDisplay\Week_2\Code_pro\python_test\StaticAnalysis.py>에서 다음 코드를 정적 분석하시오.

```
import sys
from collections import Counter

n = int(sys.stdin.readline())
cards = [int(sys.stdin.readline()) for x in range(n)]
count = Counter(sorted(cards))
print(count.most_common(1)[0][0])
```

2. 코드 품질 지표

Python 품질 측정 지표(KPI) 표

지표	목표값	측정 도구	의미
Pylint 품질 점수	$\geq 8.5 / 10$	Pylint	문법/코드 품질 준수 수준
코드 스타일 준수율	$\geq 95\%$	Flake8, Black	PEP8 준수도, 가독성 확보
타입 안정성 수준	$\geq 90\%$	Mypy / Pyright	정적 타입 검증 성공률
보안 취약점 발견 수	0건	Bandit, Safety	보안 위협 제거 여부
코드 복잡도 지수	A~B 등급	Radon, McCabe	유지보수 난이도 최소화
Dead Code 비율	$\leq 2\%$	Vulture	불필요한 코드 제거 효율
테스트 커버리지	$\geq 80\%$	pytest + coverage.py	신뢰도 높은 변경 가능성
리뷰 반영률	$\geq 95\%$	Git PR Review logs	코드 리뷰 지적사항 처리율
릴리즈 전 오류 발견율	$\geq 90\%$	CI Pipeline Logs	배포 전 품질 보증 성과
자동화된 체크 비율	$\geq 90\%$	GitHub Actions / GitLab CI	수동 오류 감소 및 정합성 확보

2. 코드 품질 지표

| 핵심 KPI 설명

품질영역	주요 KPI	기대 효과
코드 품질	Pylint 점수, Dead Code, 복잡도	유지보수 비용 ↓, 버그 ↓
표준 준수	스타일 준수율	팀 내 협업 & 리뷰 효율 ↑
안정성/보안	취약점, 타입 안정성	사고 예방
테스트	커버리지	배포 안정성 ↑
팀 프로세스	리뷰 반영, 자동화 비율	생산성 & 속도 ↑

2. 코드 품질 지표

| 추천 측정 자동화(E2E 예)

매 Pull Request 마다



정적 분석 실행



품질 점수 자동 제출



승인 전까지 기준 미달 시 병합 금지(Gate)

2. 코드 품질 지표

| 측정 가능한 품질 지표

지표	목표값	측정 도구	의미
테스트 커버리지	80% 이상	pytest-cov	테스트된 코드 비율
순환 복잡도	10 이하	radon	코드 복잡도
중복 코드율	5% 이하	CPD	중복된 코드 비율
기술 부채 비율	5% 이하	SonarQube	품질 개선 필요 코드
버그 밀도	0.5/KLOC	SonarQube	1000줄당 버그 수



3. CI/CD

CI/CD 개요

- CI/CD Continuous Integration / Continuous Deployment

개발자 커밋 → 자동 테스트 → 자동 빌드 → 자동 배포

- 장점

- 버그 조기 발견
- 배포 시간 단축
- 수동 작업 오류 감소
- 빠른 피드백

- 우리가 구축할 파이프라인

Git Push → GitHub Actions → Pytest + Lint → Render 배포



3. CI/CD

특징

워크플로우 구조

name: 워크플로우 이름

on: [트리거 이벤트]

jobs:

job-name:

runs-on: ubuntu-latest

steps:

- name: 스텝 이름

run: 실행할 명령

저장 위치

.github/

└─ workflows/

└─ ci-cd.yml



3. CI/CD

CI/CD 파이프라인 구축

● Cursor AI 프롬프트

이 FastAPI 프로젝트에 대한 GitHub Actions 워크플로를 생성합니다.

요구 사항:

1. 메인 브랜치 푸시 및 풀 리퀘스트 트리거
2. ubuntu-latest에서 실행
3. Python 3.11로 테스트
4. 필요한 단계:
 - 코드 체크아웃
 - Python 설정
 - 종속성 설치(poetry 또는 pip)
 - 커버리지를 포함한 pytest 실행
 - 린팅(ruff 또는 flake8) 실행
 - 성공 시 Render에 배포(메인 브랜치만 해당)

파일 위치: `.github/workflows/ci-cd.yml`

각 단계를 설명하는 주석을 추가합니다.



3. CI/CD

GitHub Actions 워크플로우 파일

● .github/workflows/ci-cd.yml

name: CI/CD Pipeline

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

steps:

- name: 코드 체크아웃

uses: actions/checkout@v4

- name: Python 3.11 설정

uses: actions/setup-python@v4

with:

python-version: '3.11'

- name: 의존성 캐싱

uses: actions/cache@v3

with:

path: ~/.cache/pip

key: \${{ runner.os }}-pip-\${{ hashFiles('**/requirements.txt') }}

- name: 의존성 설치

run: |

python -m pip install --upgrade pip

pip install -r requirements.txt

pip install pytest pytest-cov ruff



3. CI/CD

테스트 및 린트 단계

● ci-cd.yml (계속)

- name: 린트 체크 (Ruff)
run: |
ruff check app/ tests/
continue-on-error: false
- name: 코드 포맷 체크
run: |
ruff format --check app/ tests/
- name: 단위 테스트 실행
run: |
pytest tests/ -v --cov=app --cov-report=xml --cov-report=term
- name: 커버리지 업로드
uses: codecov/codecov-action@v3
with:
file: ./coverage.xml
fail_ci_if_error: true
- name: 테스트 결과 표시
if: always()
uses: dorny/test-reporter@v1
with:
name: Pytest Results
path: 'test-results.xml'
reporter: 'java-junit'



3. CI/CD

Render 배포 설정

● ci-cd.yml - 배포 Job

```
deploy:
  needs: test
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'

  steps:
    - name: Render 배포 트리거
      uses: johnbeynon/render-deploy-action@v0.0.8
      with:
        service-id: ${ secrets.RENDER_SERVICE_ID }
        api-key: ${ secrets.RENDER_API_KEY }

    - name: 배포 상태 확인
      run: |
        echo "Deployment triggered successfully"
        echo "Check status at: https://dashboard.render.com"

    - name: Slack 알림 (선택사항)
      if: success()
      uses: slackapi/slack-github-action@v1
      with:
        webhook-url: ${ secrets.SLACK_WEBHOOK }
        payload: |
          {
            "text": "✅ OrderBean 배포 완료!",
            "blocks": [
              {
                "type": "section",
                "text": {
                  "type": "mrkdwn",
                  "text": "*배포 성공* :rocket:\n커밋: ${ github.sha }"
                }
              }
            ]
          }
      }
```



3. CI/CD

GitHub Secrets 설정

- 민감 정보 관리

- GitHub 저장소 → Settings → Secrets → Actions
- 추가할 Secrets

```
RENDER_SERVICE_ID: srv-xxxxxxxxxxxxxx  
RENDER_API_KEY: rnd_xxxxxxxxxxxxxxxxxxxx  
DATABASE_URL: postgresql://user:pass@host:5432/db  
SECRET_KEY: your-secret-key-here
```



3. CI/CD

GitHub Secrets 설정

- Cursor AI 프롬프트로 문서화

다음에 포함하는 SECRETS.md 파일을 만드세요.

3. CI/CD에 필요한 시크릿
2. 각 시크릿을 얻는 방법(단계별)
3. GitHub에 추가하는 방법
4. 보안 모범 사례

실제 시크릿 값은 포함하지 마세요.



3. CI/CD

배포 플랫폼 선택하기

- 웹 사이트나 앱을 인터넷에 배포하려면 여러 가지 플랫폼을 선택할 수 있습니다. 대표적으로 AWS, GCP, Azure 같은 대형 클라우드 서비스와 Vercel, Netlify, Render, Railway 처럼 간단하게 설정할 수 있는 클라우드 서비스가 있습니다.
- 프론트엔드만 배포한다면 Vercel이나 Netlify, 깃허브 페이지 등을 사용할 수 있고, 풀스택 앱을 배포한다면 Render나 Railway 등을 이용하면 됩니다.



3. CI/CD

| 실습 : 배포 플랫폼 선택하기

- 풀스택 웹 개발 - OrderBean Toy Project
실습 5: 앱 배포하기 참고



3. CI/CD

| Render 플랫폼 설정

- Render.com 계정 생성 및 설정

- 회원가입: <https://render.com>
- 새 Web Service 생성
- Connect GitHub repository
- Select branch: main
- Build command: `pip install -r requirements.txt`
- Start command: `uvicorn app.main:app --host 0.0.0.0 --port $PORT`



3. CI/CD

Render 플랫폼 설정

- 환경 변수 설정

DATABASE_URL=postgresql://...

SECRET_KEY=...

ENVIRONMENT=production

- Health Check 설정

Path: /healthcheck



3. CI/CD

Health Check 엔드포인트 추가

● app/main.py에 추가

```
from fastapi import FastAPI
from datetime import datetime
import sys

app = FastAPI(title="OrderBean API")

@app.get("/healthcheck")
async def healthcheck():
    """
    서비스 상태 체크
    배포 및 모니터링 시스템이 사용
    """
    return {
        "status": "healthy",
        "timestamp": datetime.utcnow().isoformat(),
        "version": "1.0.0",
        "python_version": sys.version,
        "environment": os.getenv("ENVIRONMENT", "development")
    }

@app.get("/")
async def root():
    """루트 엔드포인트"""
    return {
        "message": "Welcome to OrderBean API",
        "docs": "/docs",
        "healthcheck": "/healthcheck"
    }
```



3. CI/CD

Health Check 엔드포인트 추가

- 테스트

```
curl http://localhost:8000/healthcheck
```



3. CI/CD

| 배포 테스트

- 코드 변경 커밋

```
git add .
```

```
git commit -m "Add CI/CD pipeline and health check"
```

```
git push origin main
```



3. CI/CD

| 배포 테스트

- GitHub Actions 확인
- GitHub 저장소 → Actions 탭
- 워크플로우 실행 상태 확인
- 각 단계별 로그 확인



3. CI/CD

배포 테스트

- Render 배포 확인

- # Render 대시보드에서 확인

- # 또는 API로 확인

- ```
curl https://your-app.onrender.com/healthcheck
```



## 3. CI/CD

### | 문제 해결

- ✖ 테스트 실패 → 로그 확인 후 수정
- ✖ 배포 실패 → Render 로그 확인
- ✔ 성공 → Production URL 테스트



## 3. CI/CD

### | 실습3 : GitHub Action

- GitHub Repository 생성 후, hello.py 생성 후 Push
- # hello.py
- `print("Hello world")`

GitHub Action with Python 테스트



# 『4과목』

## 리팩토링 + 코드 리뷰 + 배포

- Part 1. 오리엔테이션 및 목표
- Part 2. 리팩토링 이론
- Part 3. 코드 리뷰 + 배포
- Part 3. 종합 실습 & 마무리





# 1. Full-Stack Refactor + Loop Cursor AI

## 1. 현재 프로젝트 상태 확인

- `cd orderbean-project`
- `git status`
- `git log --oneline -5`



# 1. Full-Stack Refactor + Loop Cursor AI

## 2. 리팩토링 브랜치 생성

- `git checkout -b refactor/day4-improvements`



# 1. Full-Stack Refactor + Loop Cursor AI

## 3. 테스트 실행 (리팩토링 전 필수!)

- `pytest tests/ -v --cov=app`
- # 모든 테스트가 통과해야 함



# 1. Full-Stack Refactor + Loop Cursor AI

## 4. Cursor AI 준비

- Composer 모드 활성화
- 프로젝트 전체 컨텍스트 로드



# 1. Full-Stack Refactor + Loop Cursor AI

## 테스트 기반 리팩토링

### # Cursor AI 프롬프트 - Refactor 단계

## 컨텍스트

- 구현 코드: [구현 파일 경로]
- 테스트 코드: [테스트 파일 경로]
- 5단계 리뷰 결과: [개선 포인트 목록]

## 현재 코드

```
```typescript  
[현재 구현 코드]  
```
```

## 요청

다음 개선 포인트를 적용하여 리팩토링해주세요:

### 개선 포인트

1. [리뷰에서 도출된 개선점 1]
2. [리뷰에서 도출된 개선점 2]
3. [리뷰에서 도출된 개선점 3]

### 요구사항

1. \*\*기존 테스트가 모두 통과\*\*해야 함
2. 변경 사항에 대한 설명 포함
3. 필요시 새로운 테스트 추가

### 리팩토링 원칙

- DRY (Don't Repeat Yourself)
- SOLID 원칙
- React 성능 최적화 패턴



# 1. Full-Stack Refactor + Loop Cursor AI

## 다음 기능으로 반복

### # Cursor AI 프롬프트 - Loop 단계

#### ## 컨텍스트

- 완료된 기능: [방금 완료한 기능명]
- 전체 요구사항: [PRD 링크 또는 내용]

#### ## 요청

다음 작업을 분석해주세요:

#### ### 질문

1. 다음으로 구현해야 할 기능은 무엇인가요?
2. 현재 기능과의 의존성은?
3. 예상되는 테스트 케이스는?

#### ### 체크리스트

- [ ] 현재 기능 테스트 100% 통과
- [ ] 리팩토링 완료
- [ ] 문서 업데이트
- [ ] Git 커밋

#### ### 다음 단계 제안

다음 기능에 대한 Planning 단계 프롬프트를 생성해주세요.



## 2. 실습 1 - 코드 스멜 분석

### Cursor AI 프롬프트

- 예상 출력 분석
- AI가 식별할 주요 문제점들을 검토합니다.

app/services/order\_service.py를 분석하고 코드 스멜을 파악하세요.

주제:

1. 긴 메서드(20줄 이상)
2. 중복된 로직
3. 기능 선망(다른 객체의 데이터에 과도하게 접근)
4. 큰 클래스(10개 이상의 메서드)

발견된 각 코드 스멜에 대해 다음을 수행하세요.

- 정확한 위치(줄 번호) 명시
- 문제 원인 설명
- 구체적인 리팩토링 기법 제안

출력 형식:

## 코드 스멜: [유형]

위치: [파일]의 X-Y 줄

문제: [설명]

제안된 수정 사항: [리팩토링 기법]

우선순위: 높음/중간/낮음





## 2. 실습 1 - 코드 스멜 분석

### 분석 결과 예시

#### ● AI가 찾아낸 코드 스멜 예

## 코드 냄새: Long 메소드

위치: order\_service.py의 45-89행

메소드: create\_order()

문제: 45줄의 방법이 검증, 작업, 저장 등의 의미 있는 책임 수행

제안된 수정 사항: 추출 방법

우선순위: 높음

## 코드 냄새: 중복된 논리

위치:

- order\_service.py의 92-98행

- order\_service.py의 145-151행

문제: 피드백이 있습니다.

제안된 수정 사항: 일반적인 유효성 검사 방법 추출

우선순위: 중간

## 코드 냄새: 기능 부러움

위치: order\_service.py의 110-120행

문제: 주문이 사용자의 속성을 과도하게 접근하게 합니다

제안된 수정 사항: 메서드를 User 클래스로 이동

우선순위: 중간



## 2. 실습 1 - 코드 스멜 분석

### 리팩토링 전략 🛠️

- 코드 스멜(Code Smell) 감지

| 코드 스멜          | 설명               | 해결 방법           |
|----------------|------------------|-----------------|
| Long Method    | 50줄 이상의 긴 메서드    | 작은 메서드로 분리      |
| Large Class    | 너무 많은 책임을 가진 클래스 | 클래스 분할 (SRP)    |
| Duplicate Code | 중복된 코드           | 공통 함수로 추출       |
| Magic Number   | 의미 없는 숫자 리터럴     | 상수로 정의          |
| Deep Nesting   | 5단계 이상 중첩        | Early Return 사용 |
| God Object     | 모든 걸 아는 객체       | 책임 분산           |

# 3. 실습 2 - Extract Method 리팩토링

## | 현재 코드 (order\_service.py)

```
def create_order(self, order_data: dict):
 # 긴 메서드 (45줄)

 # 검증 로직 (10줄)
 if not order_data.get('user_id'):
 raise ValueError("user_id required")
 if not order_data.get('items'):
 raise ValueError("items required")
 # ... 더 많은 검증

 # 계산 로직 (15줄)
 total = 0
 for item in order_data['items']:
 total += item['price'] * item['quantity']
 # ... 할인 계산

 # 저장 로직 (20줄)
 order = Order(**order_data)
 self.db.add(order)
 self.db.commit()
 # ... 추가 처리

 return order
```

# 3. 실습 2 - Extract Method 리팩토링

## | Extract Method - Cursor AI 실습

### 프롬프트 1: 검증 로직 추출

`create_order()`에서 검증 로직을 별도의 메서드로 추출합니다.

요구 사항:

1. 메서드 이름: `_validate_order_data()`
2. 매개변수: `order_data(dict)`
3. 명확한 메시지와 함께 `ValueError` 발생
4. 검증 규칙을 설명하는 `docstring` 추가
5. 기존 테스트는 모두 통과 상태로 유지

추출 후, 새 메서드를 사용하도록 `create_order()`를 업데이트합니다.

# 3. 실습 2 - Extract Method 리팩토링

## Extract Method - Cursor AI 실습

### Cursor가 생성할 코드 예

```
def __validate_order_data(self, order_data: dict) -> None:
```

```
 """
 주문 데이터 유효성 검증
```

```
 Args:
```

```
 order_data: 주문 정보 딕셔너리
```

```
 Raises:
```

```
 ValueError: 필수 필드 누락 또는 잘못된 값
```

```
 """
```

```
 required_fields = ['user_id', 'items']
```

```
 for field in required_fields:
```

```
 if not order_data.get(field):
```

```
 raise ValueError(f"{field} is required")
```

```
 if not isinstance(order_data['items'], list):
```

```
 raise ValueError("items must be a list")
```

```
 if len(order_data['items']) == 0:
```

```
 raise ValueError("items cannot be empty")
```

# 3. 실습 2 - Extract Method 리팩토링

## Extract Method - 계산 로직 분리

### 프롬프트 2

가격 계산 로직을 별도의 메서드로 추출합니다.

요구 사항:

1. 메서드 이름: `_calculate_order_total()`
2. 매개변수: `items`(목록)
3. 반환값: `float`(총액)
4. 할인 로직을 별도로 처리합니다.
5. 이 메서드에 대한 단위 테스트를 추가합니다.

메서드를 생성하고 `create_order()`를 적절히 업데이트합니다.

# 3. 실습 2 - Extract Method 리팩토링

## Extract Method - 계산 로직 분리

### 결과 코드 예

```
def _calculate_order_total(self, items: list) -> float:
```

```
 """
 주문 총액 계산
```

```
 Args:
```

```
 items: 주문 항목 리스트
```

```
 Returns:
```

```
 총 금액
```

```
 """
```

```
 subtotal = sum(
 item['price'] * item['quantity']
 for item in items
)
 return subtotal
```

```
def _apply_discount(self, amount: float, discount_code: str = None) -> float:
```

```
 """할인 적용"""
```

```
 if not discount_code:
 return amount
```

```
 discount_rate = self._get_discount_rate(discount_code)
 return amount * (1 - discount_rate)
```

# 3. 실습 2 - Extract Method 리팩토링

## 리팩토링 후 create\_order 메서드 개선된 코드 예

```
def create_order(self, order_data: dict) -> Order:
 """
 주문 생성

 Args:
 order_data: 주문 정보

 Returns:
 생성된 주문 객체
 """
 # 명확한 단계별 처리
 self._validate_order_data(order_data)

 subtotal = self._calculate_order_total(order_data['items'])
 total = self._apply_discount(
 subtotal,
 order_data.get('discount_code')
)

 order = self._create_order_entity(order_data, total)
 self._save_order(order)

 return order
```



### 3. 실습 2 - Extract Method 리팩토링

## 리팩토링 후 create\_order 메서드 개선 효과

- 45줄 → 15줄 (67% 감소)
- 각 단계가 명확함
- 테스트 가능한 작은 단위

# 3. 실습 2 - Extract Method 리팩토링

## | 테스트 실행 및 검증

### 리팩토링 후 필수 검증

# 1. 전체 테스트 실행

```
pytest tests/ -v
```

# 2. 커버리지 확인

```
pytest --cov=app --cov-report=html
```

# 3. 결과 확인

# All tests passed

Coverage maintained or improved

# 3. 실습 2 - Extract Method 리팩토링

## | 테스트 실행 및 검증

리팩토링실패 시 대응 후 필수 검증

황금 규칙

"테스트가 실패하면 리팩토링을 커밋하지 마세요!"

# 테스트 실패하면 즉시 롤백

git diff # 변경 내용 확인

git checkout -- app/services/order\_service.py # 되돌리기

# 또는 stash 후 재시도

git stash



## 4. 실습 3 - 중복 코드 제거

### AI가 찾을 중복 패턴

#### Cursor AI 프롬프트

order\_service.py 전체에서 중복된 검증 로직을 찾아 제거합니다.

단계:

1. 유사한 검증이 발생하는 모든 위치를 파악합니다.
2. 공통 검증 유틸리티를 만듭니다.
3. 모든 중복 코드를 해당 유틸리티로 바꿉니다.
4. 모든 테스트가 여전히 통과하는지 확인합니다.
5. 필요한 경우 새 유틸리티에 대한 테스트를 추가합니다.

먼저 중복된 코드를 보여준 다음 리팩토링된 버전을 보여줍니다.



## 4. 실습 3 - 중복 코드 제거

### I AI가 찾을 중복 패턴

### AI가 찾을 중복 패턴 결과 예

```
create_order()에서
if not order_data.get('user_id'):
 raise ValueError("user_id required")

update_order()에서
if not order_data.get('user_id'):
 raise ValueError("user_id required")

cancel_order()에서
if not order_id:
 raise ValueError("order_id required")
```



## 4. 실습 3 - 중복 코드 제거

### 중복 제거 - 공통 유틸리티 생성

### 새로운 파일: app/utils/validators.py 예

```
from typing import Any, List

class ValidationError(Exception):
 """검증 실패 예외"""
 pass

def require_fields(data: dict, fields: List[str]) -> None:
 """
 필수 필드 검증

 Args:
 data: 검증할 데이터
 fields: 필수 필드 리스트

 Raises:
 ValidationError: 필수 필드 누락
 """
 missing = [f for f in fields if not data.get(f)]
 if missing:
 raise ValidationError(
 f"Required fields missing: {' '.join(missing)}"
)

def validate_positive_amount(amount: float, field_name: str = "amount") -> None:
 """양수 금액 검증"""
 if amount <= 0:
 raise ValidationError(f"{field_name} must be positive")

def validate_non_empty_list(items: list, field_name: str = "items") -> None:
 """빈 리스트 검증"""
 if not isinstance(items, list) or len(items) == 0:
 raise ValidationError(f"{field_name} cannot be empty")
```



## 4. 실습 3 - 중복 코드 제거

### 공통 유틸리티 적용

#### Before (중복 코드)

```
order_service.py
def create_order(self, order_data: dict):
 if not order_data.get('user_id'):
 raise ValueError("user_id required")
 if not order_data.get('items'):
 raise ValueError("items required")
 # ...

def update_order(self, order_id: int, order_data: dict):
 if not order_data.get('user_id'):
 raise ValueError("user_id required")
 if not order_data.get('items'):
 raise ValueError("items required")
 # ...
```



## 4. 실습 3 - 중복 코드 제거

### 공통 유틸리티 적용

#### After (유틸리티 사용)

```
from app.utils.validators import require_fields, validate_non_empty_list
```

```
class OrderService:
```

```
 def create_order(self, order_data: dict):
 require_fields(order_data, ['user_id', 'items'])
 validate_non_empty_list(order_data['items'])
 # ...
```

```
 def update_order(self, order_id: int, order_data: dict):
 require_fields(order_data, ['user_id', 'items'])
 validate_non_empty_list(order_data['items'])
 # ...
```





## 5. 실습 4 - SOLID 원칙 적용



### SRP

## Single Responsibility Principle 위반 사례

```
class OrderService:
 """너무 많은 책임을 가진 클래스"""

 def create_order(self): ... # 주문 관리
 def calculate_total(self): ... # 가격 계산
 def process_payment(self): ... # 결제 처리
 def send_email(self): ... # 알림 발송
 def generate_invoice(self): ... # 문서 생성
```



# 5. 실습 4 - SOLID 원칙 적용

## SRP

### Cursor AI 프롬프트

단일 책임 원칙을 따르도록 OrderService를 리팩토링합니다.

현재 문제:

- OrderService는 주문 관리, 가격 책정, 결제 및 알림을 처리합니다.
- 클래스가 400줄이 넘습니다.

작업:

1. 가격 책정 로직을 위한 PriceCalculator 추출
2. 결제 처리를 위한 PaymentService 추출
3. 이메일/SMS를 위한 NotificationService 추출
4. 종속성 주입을 통해 이러한 서비스를 사용하도록 OrderService 업데이트
5. 새로운 구조에서 작동하도록 모든 테스트 업데이트

전환하는 동안 이전 버전과의 호환성 유지



# 5. 실습 4 - SOLID 원칙 적용



## SRP 적용 - 서비스 분리 새로운 구조 예

```
app/services/price_calculator.py
class PriceCalculator:
 """가격 계산 전담"""

 def calculate_subtotal(self, items: list) -> float:
 return sum(item['price'] * item['quantity'] for item in items)

 def apply_discount(self, amount: float, code: str) -> float:
 rate = self._get_discount_rate(code)
 return amount * (1 - rate)

 def add_shipping_fee(self, amount: float) -> float:
 return amount + 3000 if amount < 50000 else amount

app/services/payment_service.py
class PaymentService:
 """결제 처리 전담"""

 def process_payment(self, order_id: int, amount: float, method: str):
 # 결제 처리 로직
 pass

 def validate_payment_method(self, method: str) -> bool:
 return method in ['card', 'bank', 'mobile']

app/services/notification_service.py
class NotificationService:
 """알림 발송 전담"""

 def send_order_confirmation(self, order: Order):
 self._send_email(order.user.email, "주문 확인", ...)

 def send_shipping_notification(self, order: Order):
 self._send_sms(order.user.phone, "배송 시작", ...)
```



# 5. 실습 4 - SOLID 원칙 적용



## Dependency Injection 적용 개선된 OrderService 예

```
class OrderService:
 """의존성 주입을 통한 협력"""

 def __init__(
 self,
 db: Database,
 price_calculator: PriceCalculator,
 payment_service: PaymentService,
 notification_service: NotificationService
):
 self.db = db
 self.price_calculator = price_calculator
 self.payment_service = payment_service
 self.notification_service = notification_service

 def create_order(self, order_data: dict) -> Order:
 """명확한 역할 분담"""
 # 1. 검증
 self._validate_order_data(order_data)

 # 2. 가격 계산 (위임)
 total = self.price_calculator.calculate_total(order_data)

 # 3. 주문 생성
 order = Order(**order_data, total=total)
 self.db.add(order)
 self.db.commit()

 # 4. 결제 처리 (위임)
 self.payment_service.process_payment(order.id, total)

 # 5. 알림 발송 (위임)
 self.notification_service.send_order_confirmation(order)

 return order
```

## 9. Day 4 학습 완료

### | 오늘 배운 것

- 리팩토링 이론과 코드 스멜 식별
- AI를 활용한 실전 리팩토링
- SOLID 원칙 적용
- GitHub Actions CI/CD 구축
- Render 플랫폼 배포

# THANK YOU.

앞으로의 엔지니어는 단순한 '코더'나 '기계 조작자'가 아니라 뇌-기계 인터페이스를 통해 지식과 능력을 즉각 확장하는 존재(뉴로-인터페이스: Neuro Interface)가 될 수 있습니다.

- 🎯 목표 달성을 위한 여정이 시작됩니다.
- 🌟 궁금한 점이 있으시면 언제든지 문의해주세요!
- 🚀 함께 코더와 프롬프트 전문가로 성장해 나갑시다!